

Licenciatura em Engenharia Informática

Escola Superior de Tecnologia e Gestão

Instituto Politécnico de Viana do Castelo

Unidade Curricular de Projeto II

Relatório de projeto
2025/2026

Plataforma de Gestão de Horários por Turnos

Alunos: 33400 - Francisco Gomes e 33397 - Tiago Costa

Resumo

O presente relatório descreve o processo de análise, desenho e implementação da Plataforma de Gestão de Horários por Turnos, desenvolvida para apoiar a operação da loja Levi's Braga Parque. O projeto surgiu da necessidade de substituir o planeamento manual de escalas por uma solução digital capaz de centralizar horários, preferências, folgas, permutas e decisões de gestão.

A metodologia adotada seguiu as melhores práticas de Engenharia de Software. Numa primeira fase, procedeu-se ao levantamento de requisitos, modelação de processos (BPMN), especificação técnica (UML) e implementação de uma base de dados relacional em PostgreSQL.

Numa segunda fase, esses modelos foram materializados numa aplicação desktop desenvolvida com a framework Java Spring Boot. Implementou-se uma arquitetura em camadas, separando o mapeamento objeto-relacional (JPA/Hibernate), a Camada de Acesso a Dados (DAL/Repositories) e a Camada de Lógica de Negócio (BLL).

A solução final implementa autenticação, dashboard por perfil, gestão da loja, gestão de funcionários, preferências do funcionário com aprovação pela gerência, pedidos de folga, permutas, geração de propostas mensais de horário, validação pelo supervisor, relatórios mensais e empacotamento ZIP para entrega. A aplicação foi preparada com dados de demonstração e documentação de apoio, permitindo uma apresentação estável e coerente com os requisitos definidos.

4.4.1.6	Caso de Uso: Listar pedidos permutas de horário	35
4.4.1.7	Caso de Uso: Lista de preferências	36
4.4.1.8	Caso de Uso: Pedir troca de turno	37
4.4.1.9	Caso de Uso: Validar Proposta de Horário	38
4.4.2	Especificação usando Diagramas de atividades	39
4.4.2.1	Diagrama de Atividades: Gerar Horário Automático (UC-S01)	39
4.4.2.2	Diagrama de Atividades: Selecionar e aprovar/recusar permutas (UC-G05a)	40
4.4.2.3	Diagrama de Atividades: Inserir preferência de dias de folga (UC-F01a)	41
4.4.2.4	Diagrama de Atividades: Inserir preferências de época de férias (UC-F01d)	42
4.4.2.5	Diagrama de Atividades: Aprovar/Recusar as preferências (UC-G03c)	43
4.4.2.6	Diagrama de Atividades: Qtd. de funcionários a trabalhar simultaneamente (UC-G04a)	44
4.4.2.7	Diagrama de Atividades: Inserir preferências de colegas de trabalho (UC-F01c)	45
4.4.2.8	Diagrama de Atividades: Definir dia do mês que o horário será lançado (UC-G04b)	46
4.4.3	Especificação usando Diagramas de sequência	47
4.4.3.1	Caso de Uso: Visualizar Horário (UC-F03)	47
4.4.3.2	Caso de Uso: Adicionar Funcionário (UC-G02a)	48
5	Desenho e Implementação da BD	49
5.1	Modelação de Dados	49
5.1.1	Estrutura de tabelas inicial (Pré-Normalização)	50
5.1.2	Estrutura de tabelas final (Pós-Normalização)	51
5.2	Modelo Físico da Base de dados	52
5.3	Tecnologias usadas e Criação da Base de Dados	53
5.4	Queries à Base de Dados	57
6	Arquitetura e Implementação do Sistema (Projeto II)	60
6.1	Arquitetura em Camadas	60
6.2	Mapeamento Objeto-Relacional (JPA/Hibernate)	60
6.3	Camada de Lógica de Negócio (BLL)	61
6.3.1	Exemplo de validação de autenticação	61
6.3.2	Principais fluxos implementados	61
6.4	Segurança e auditoria	62
7	Implementação da Interface Desktop (Fase 2)	62
7.1	Transição para Interface Gráfica	62
7.2	Tecnologias de Prototipagem e Design	62
7.3	Mockups e Protótipos de Ecrã	63
7.3.1	Ecrã de Autenticação (Login)	63
7.3.2	Painel do Gerente (<i>Manager Dashboard</i>)	64

7.3.3	Painel do Funcionário (<i>Employee Dashboard</i>)	65
7.4	Implementação funcional da aplicação desktop	66
7.5	Integração entre interface, lógica e dados	66
8	Implementação da Plataforma Web (Fase 3)	67
8.1	Interface do Funcionário – Módulo Complementares	67
8.2	Interface do Funcionário – Módulo Horários	68
8.3	Arquitetura de Fluxo de Dados – UC-F01c e UC-F03	69
8.3.1	UC-F01c – Preferências de Colegas de Trabalho	69
8.3.2	UC-F03 – Visualização do Calendário de Horários	71
8.4	Hardening de Segurança e Consistência Pós-Integração Multi-Loja	73
8.4.1	Correção de Controlo de Acesso no WebGuardInterceptor	73
8.4.2	Alinhamento de Esquema da Base de Dados e Testes de Integração	74
8.4.3	Gestão Multi-Loja e Consistência de Navegação: o enum ContextoSelecao	74
8.4.4	Pipeline de Validação da Configuração de Horário da Loja	75
8.4.5	Outros Aspetos de Implementação: Concorrência na UI, Trata- mento de Erros e Controlo de Acesso	76
9	Conclusão e Trabalho Futuro	78
9.1	Conclusão	78
9.2	Trabalho Futuro	79

Índice de Figuras

1	Modelação dos processos de negócio do sistema.	12
2	Processo 1 - Definir Horário de Funcionários.	13
3	Processo 2 - Definir Preferências.	14
4	Processo 3 - Permuta de Horários.	15
5	Processo 4 - Registar Funcionários.	16
6	Subprocesso de Registar Funcionários.	16
7	Processo 5 - Definir parâmetros da loja.	17
8	Subprocesso de Registar Funcionários.	17
9	Processo 6 - Alterar Horário da Loja para Épocas Especiais.	18
10	Processo 7 - Alterar Dados ou Ativar/Desativar Funcionários.	19
11	<i>Modelo de Casos de Uso.</i>	26
12	<i>Modelo de Classes de Domínio.</i>	27
13	<i>Diagrama de Transição de Estados - Utilizador.</i>	28
14	<i>Diagrama de Transição de Estados - Horário.</i>	28
15	<i>Diagrama de Transição de Estados - Permuta.</i>	29
16	<i>Diagrama de Transição de Estados - DayOff.</i>	29
17	Diagrama de atividades para o caso de uso <i>Gerar Horário Automático</i> . Mostra o processo de geração automática do horário mensal, incluindo validações das regras do sistema (RFS01-09) e notificações aos funcionários.	39
18	Diagrama de atividades para o caso de uso <i>Selecionar e aprovar/recusar permutas</i> . Ilustra o processo de análise e decisão do gerente sobre pedidos de troca de turnos entre funcionários, incluindo a verificação de viabilidade (RFU06).	40
19	Diagrama de atividades para o caso de uso <i>Inserir preferência de dias de folga</i> . Mostra o processo de um funcionário submeter os seus dias preferi- dos para folga, incluindo validações de limites (RFS04) e aprovação pelo gerente (RFU07).	41
20	Diagrama de atividades para o caso de uso <i>Inserir preferências de época de férias</i> . Ilustra o processo de submissão de períodos preferidos para férias, incluindo validações de disponibilidade e conflitos com outros pedidos.	42
21	Diagrama de atividades para o caso de uso <i>Aprovar/Recusar as preferên- cias</i> . Mostra o processo de análise e decisão do gerente sobre as preferências submetidas pelos funcionários (RFU07).	43
22	Diagrama de atividades para o caso de uso <i>Quantidade de funcionários a trabalhar simultaneamente</i> . Ilustra a definição da regra de cobertura mínima por turno (RFU08, RFS05).	44
23	Diagrama de atividades para o caso de uso <i>Inserir preferências de colegas de trabalho</i> . Mostra o processo de um funcionário definir os colegas com quem prefere trabalhar, incluindo validações de disponibilidade e aprova- ção (RFU10).	45
24	Diagrama de atividades para o caso de uso <i>Definir dia do mês que o horário será lançado</i> . Mostra o processo de configuração da data de publicação automática do horário mensal (RFU09, RFS08).	46
25	Diagrama de sequência do caso de uso UC-F03: Visualizar Horário	47
26	Diagrama de sequência do caso de uso UC-G02a: Adicionar Funcionário	48
27	Modelo Físico da Base de Dados (DER).	52

28	Protótipo de Alta Fidelidade - Ecrã de Login da Aplicação Desktop. . . .	63
29	Protótipo de Alta Fidelidade - Painel do Gerente (Dashboard Operacional). 64	
30	Protótipo de Alta Fidelidade - Painel do Funcionário com listagem de turnos e atalhos de gestão.	65

Índice de Tabelas

1	Tabela de Requisitos Funcionais de Utilizador	21
2	Tabela de Requisitos Funcionais do Sistema	23
3	Tabela de Requisitos Não Funcionais	24
4	Caso de Uso UC-G01: Definir horário de funcionamento da loja	30
5	Caso de Uso UC-G01a: Alterar horário para épocas especiais	31
6	Caso de Uso UC-G03a: Alterar funcionário	32
7	Caso de Uso UC-G03b: Desativar funcionário	33
8	Caso de Uso UC-G04: Definir regras do sistema	34
9	Caso de Uso UC-G05: Listar pedidos permutas	35
10	Caso de Uso UC-F01: Lista de preferências	36
11	Caso de Uso UC-F02: Pedir troca de turno	37
12	Caso de Uso UC-S01: Validar Proposta de Horário	38
13	Endpoints do WebComplementaresController (base: /web/complementares)	67
14	Endpoints do WebHorariosController (base: /web/horarios)	68

Índice de Código

1	Criação de Tipos e Tabelas Principais	54
2	Tabelas de Associação e Dependentes	55
3	Índices de Performance	56
4	Query para obter o horário mensal de um funcionário	57
5	Query para listar funcionários escalados numa data específica	57
6	Contagem de turnos por funcionário para controlo de equidade	58
7	Listagem de permutas pendentes de aprovação	58
8	Query para Supervisor listar horários pendentes de aprovação	59
9	Exemplo simplificado de autenticação e validação de utilizador ativo . . .	61
10	Correção do bloqueio de acesso por URL no WebGuardInterceptor . . .	73
11	Migração idempotente adicionada para sincronizar o esquema com a entidade Turno	74
12	Enum ContextoSelecao – origem da navegação para o ecrã de seleção de loja	75
13	Verificação da cobertura exata dos extremos de abertura/fecho	76

1 Introdução

1.1 Apresentação do tema

Este projeto tem como principal objetivo o desenvolvimento de um software de gestão de horários para empresas. O sistema visa otimizar o processo da criação de horários e a organização por turnos, bem como a comunicação interna entre gestores e colaboradores.

Atualmente, o planeamento manual de horários é um grande desafio comum em ambientes de trabalho, sobretudo em lojas com equipas rotativas e diferentes tipos de turnos, sejam eles a tempo inteiro, part-time ou intermediário. Essa prática tende, por vezes, a gerar conflitos de disponibilidade, sobreposição de horários e falha na distribuição equitativa do trabalho.

1.2 Objetivos do projeto

Com o desenvolvimento do sistema, pretende-se otimizar a criação, a gestão e a visualização dos horários garantindo:

- A eficiência e rapidez na elaboração dos planos semanais/mensais;
- Transparência e acessibilidade para todos os colaboradores;
- Redução de erros humanos na criação manual dos horários e na duplicação de turnos;
- Maior controlo das folgas, férias, substituições e ausências.

O projeto insere-se no âmbito de uma proposta tecnológica para a digitalização de processos internos da loja Levi's Braga Parque, podendo também ser aplicado a outras empresas que necessitem deste tipo de serviço, contribuindo para uma gestão moderna e sustentável de recursos humanos.

2 Apresentação do negócio

O projeto "Levi's Braga Parque - Plataforma de Gestão de Horários por Turnos" tem como objetivo otimizar o planeamento e a organização dos horários de trabalho da equipa da loja Levi's no centro comercial Braga Parque.

O sistema suporta a gestão de recursos humanos de forma automatizada, permitindo:

- Criação automatizada de escalas e turnos;
- Consideração das disponibilidades e preferências de cada funcionário;
- Gestão de férias, aniversários e outras datas festivas;
- Controlo de trocas de turnos entre colaboradores.

Desta forma pretende-se reduzir o esforço administrativo, aumentar a eficiência e garantir uma comunicação clara e transparente entre gerentes e funcionários.

2.1 Âmbito de enquadramento do negócio

O sistema é direcionado a três perfis de utilizadores, refletindo a hierarquia da loja:

- **Gerente:** Responsável pela gestão global do sistema, incluindo a definição de regras, criação de escalas e gestão de dados (funcionários e lojas).
- **Supervisor:** Atua como um nível intermédio de validação, sendo responsável por analisar e aprovar a proposta de horário gerada pelo sistema antes da sua publicação oficial para a equipa.
- **Funcionário:** Utilizador final que consulta o seu horário publicado, regista as suas preferências (disponibilidade, folgas) e pode solicitar trocas de turnos.

O software está inserido no contexto de gestão interna da loja, apoiando diretamente a área de Recursos Humanos e centralizando toda a informação sobre os horários e turnos num único sistema, garantindo um fluxo de aprovação estruturado entre a gerência e os colaboradores.

Principais Processos de Negócio Identificados:

1. Criação e gestão de horários (automática);
2. Consulta e confirmação de turnos;
3. Pedido de troca de turnos;
4. Gestão de férias, datas festivas e ausências.

Cada processo interage com entidades do sistema e segue regras de validação automática para garantir a integridade e eficiência.

2.2 Modelação dos Processos de Negócio

A modelação dos processos de negócio do sistema tem como objetivo representar graficamente e de forma estruturada o modo como as diferentes atividades da loja se relacionam entre si, no contexto da gestão de horários e da organização da equipa de trabalho.

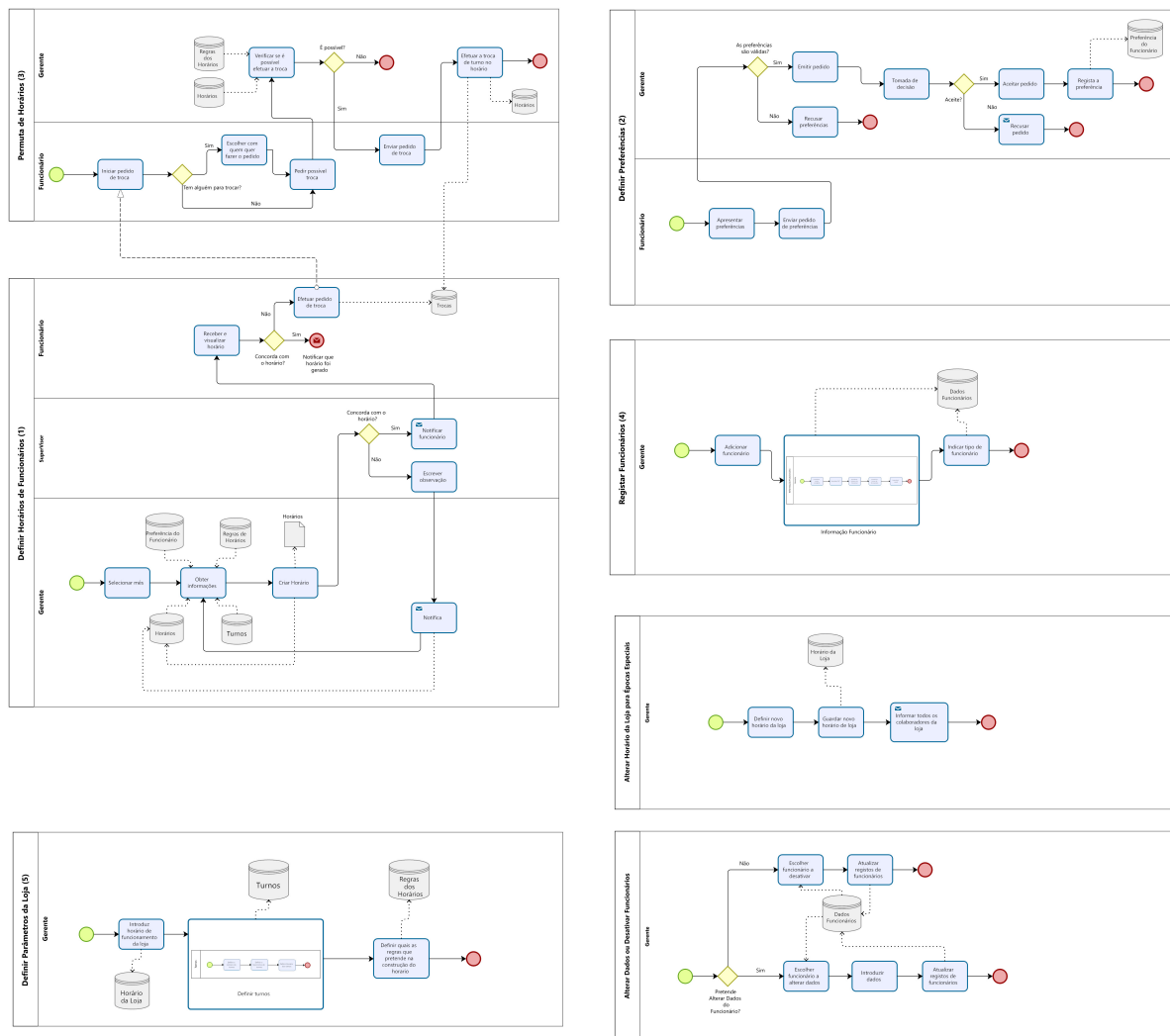


Figura 1: Modelação dos processos de negócio do sistema.

2.2.1 Processo 1 - Definir Horário de Funcionários

Neste processo, o sistema gera automaticamente a proposta de horário de trabalho mensal ou semanal dos colaboradores. A definição da escala baseia-se nas informações registadas sobre os funcionários — como as suas disponibilidades e preferências — e nas regras definidas pela loja (como folgas obrigatórias, número mínimo de funcionários por turno e outras restrições legais).

O fluxo contempla um nível de validação hierárquica: após a criação da proposta pelo Gerente, esta é submetida à análise do Supervisor. Caso o Supervisor concorde com a distribuição, o horário é aprovado e notificado aos funcionários; caso contrário, é devolvido ao Gerente com observações para retificação. Após a publicação final, os funcionários podem consultar a sua escala e, se necessário, efetuar pedidos de permuta de turno.

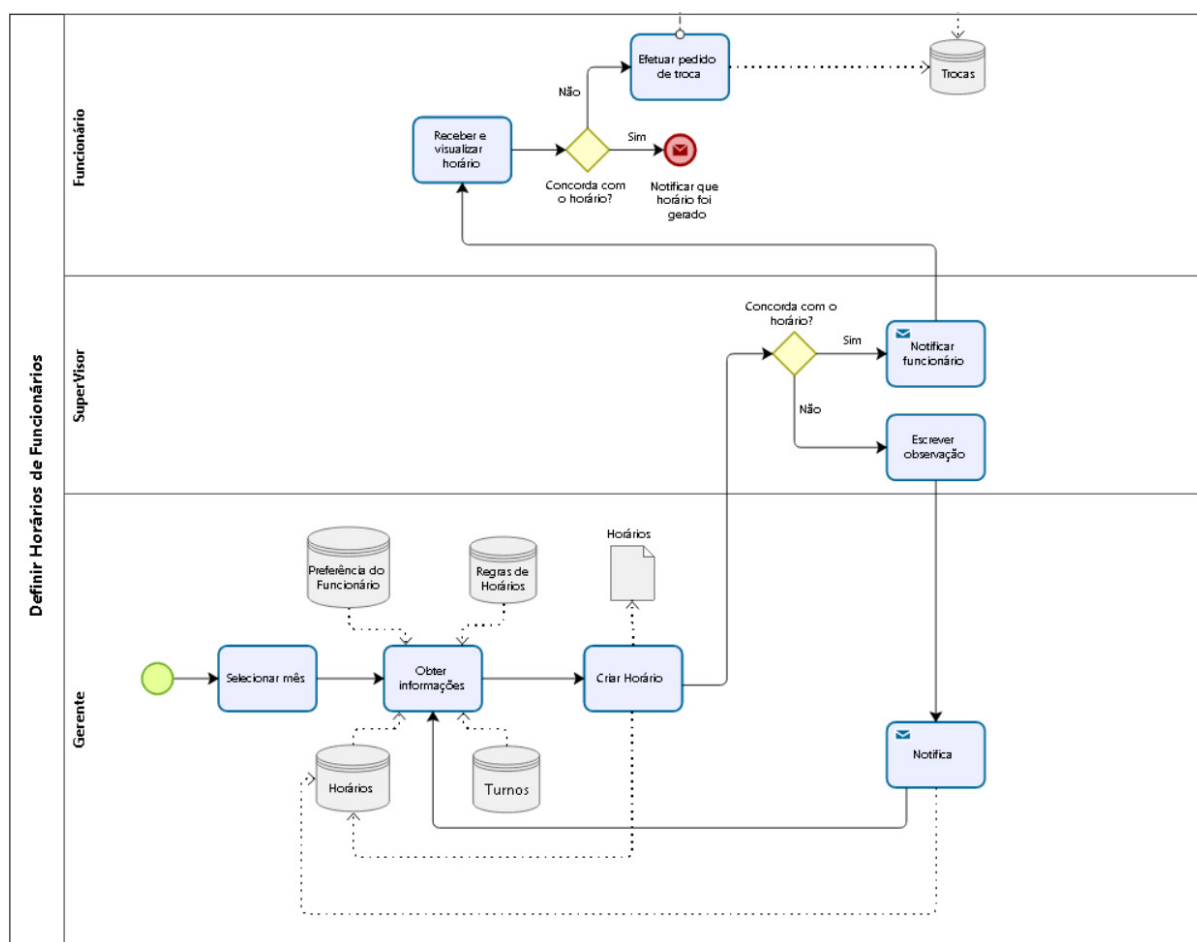


Figura 2: Processo 1 - Definir Horário de Funcionários.

2.2.2 Processo 2 - Definir Preferências

Cada funcionário pode indicar as suas preferências de trabalho, como turnos de maior conveniência (manhã, tarde ou noite), colegas com quem trabalha melhor, folgas e férias. Estas preferências são levadas em consideração pelo sistema durante a criação automática do horário, promovendo maior satisfação e produtividade da equipa.

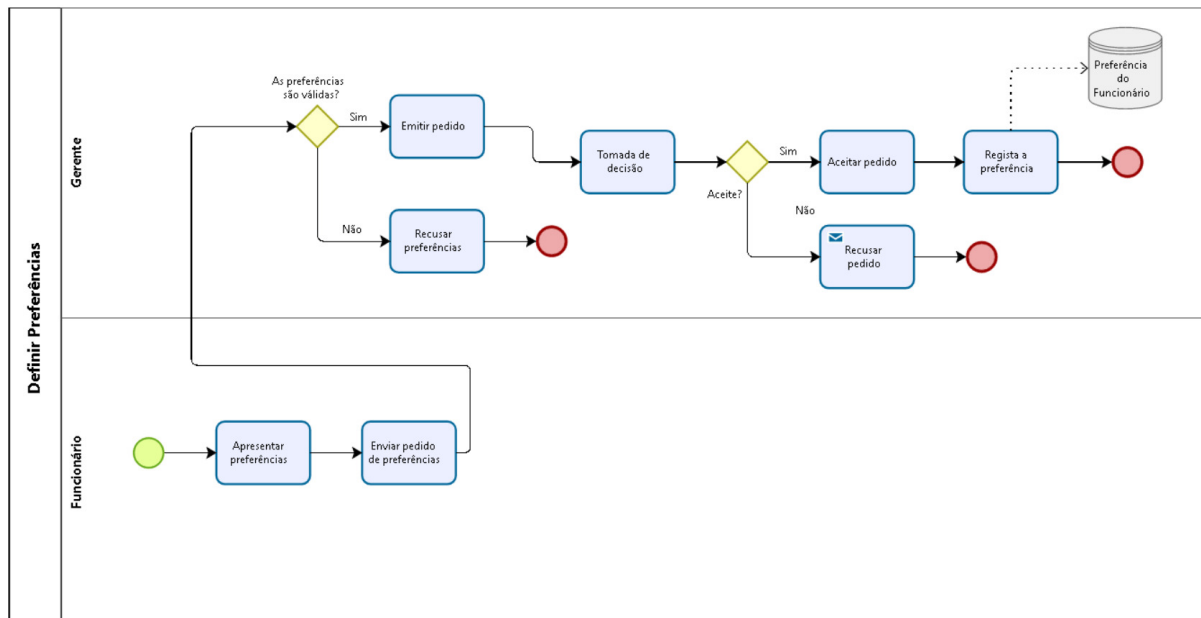


Figura 3: Processo 2 - Definir Preferências.

2.2.3 Processo 3 - Permuta de Horários

Este processo permite que dois funcionários troquem turnos entre si de forma simples e controlada. A permuta é iniciada por um dos funcionários, confirmada pelo outro e validada pelo gerente. O sistema atualiza automaticamente o horário após a aprovação, mantendo o histórico das alterações de turnos efetuadas.

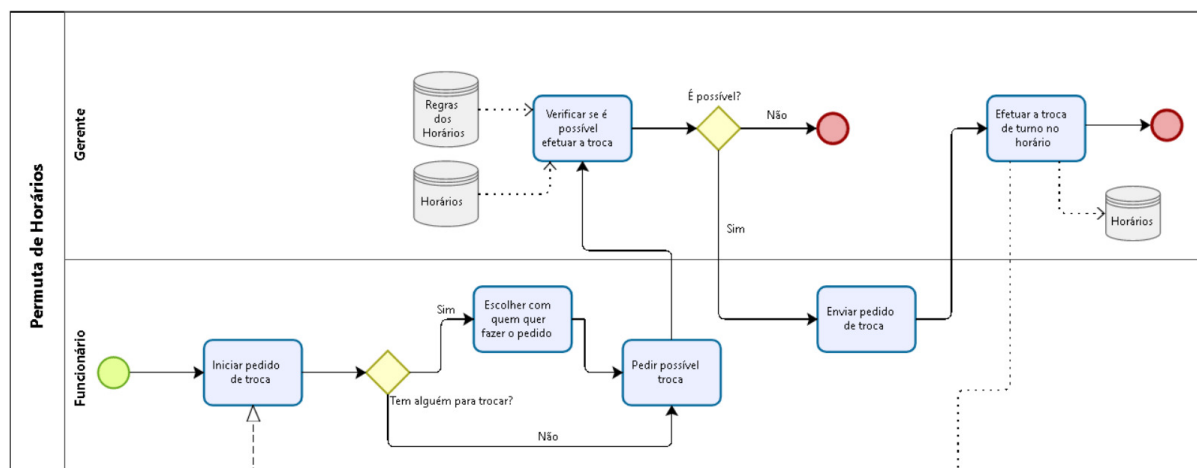


Figura 4: Processo 3 - Permuta de Horários.

2.2.4 Processo 4 - Registar Funcionários

Corresponde ao processo de criação e gestão dos perfis de cada colaborador. O gerente regista as informações essenciais, como o nome (e outros dados pessoais), a sua função, disponibilidade, preferências e folgas. Estes dados servem de base para todos os outros processos do sistema, principalmente no processo da criação automática dos horários.

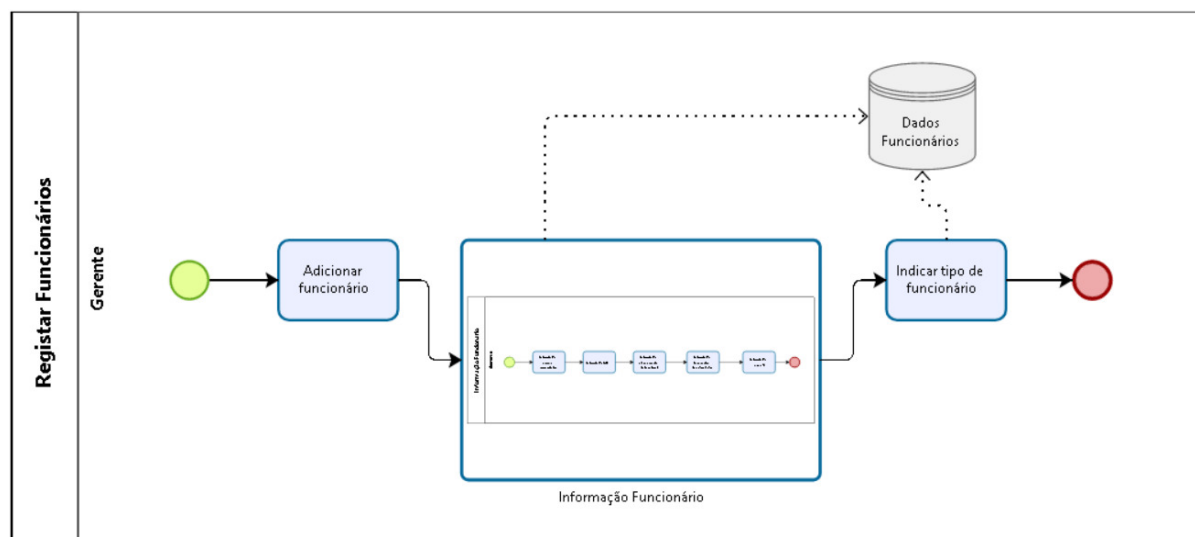


Figura 5: Processo 4 - Registar Funcionários.

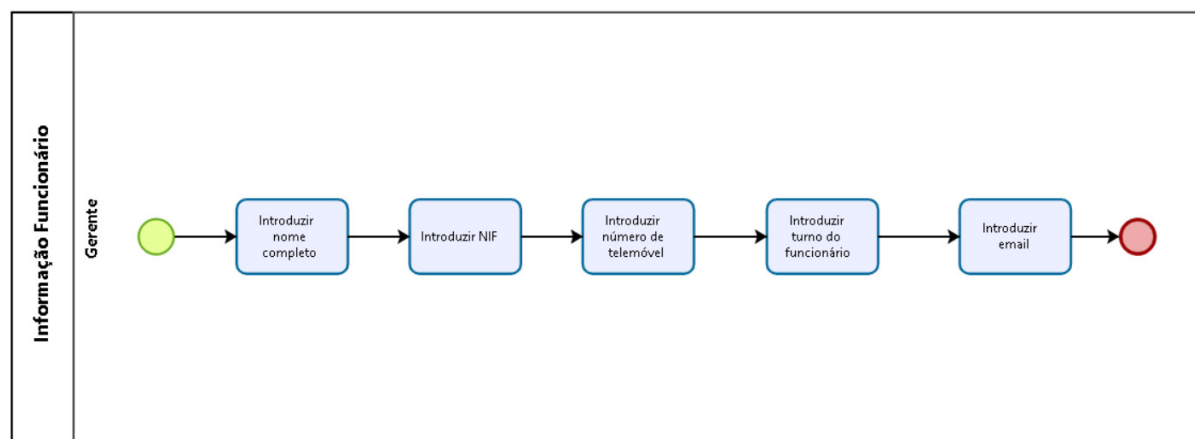


Figura 6: Subprocesso de Registar Funcionários.

2.2.5 Processo 5 - Definir parâmetros da loja

Neste processo, o gerente estabelece as regras e políticas internas que o sistema deve seguir durante a criação dos horários, como :

- o número mínimo de funcionários por turno;
- a obrigatoriedade de certos cargos (ex: Gerente e Sub-Gerente) trabalharem aos sábados;
- a duração máxima dos turnos;
- as regras legais e internas (como folgas obrigatórias e os períodos de descanso), entre outros;

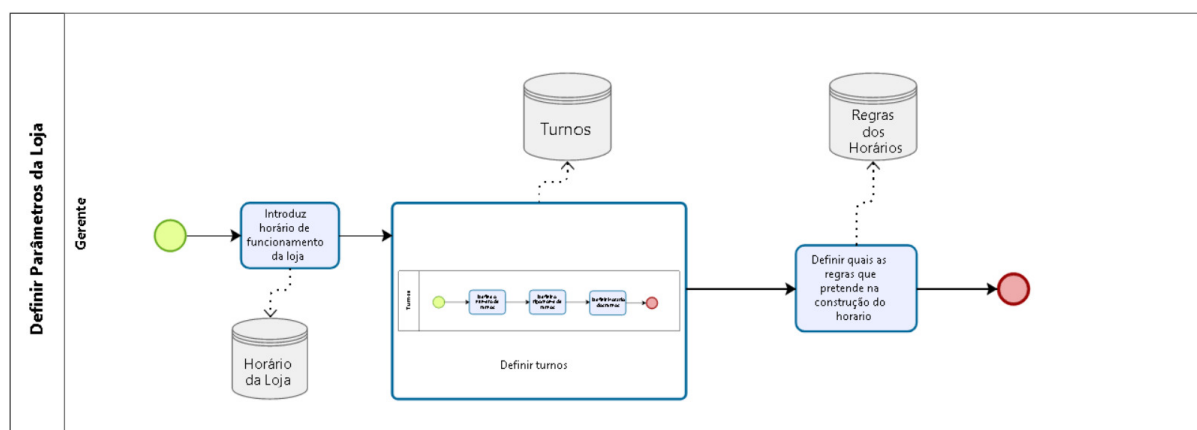


Figura 7: Processo 5 - Definir parâmetros da loja.

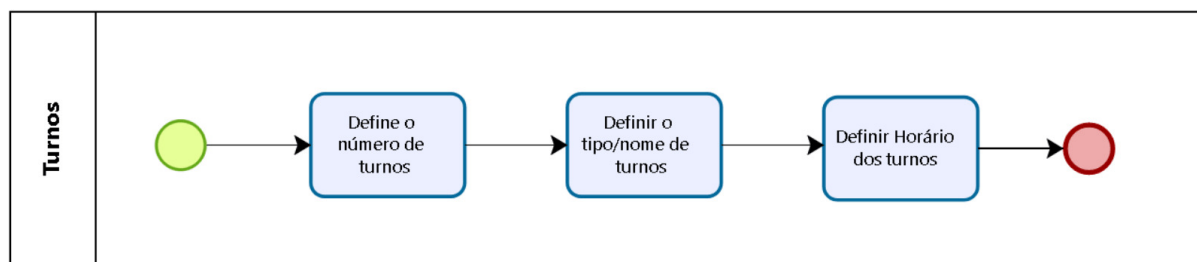


Figura 8: Subprocesso de Registrar Funcionários.

2.2.6 Processo 6 - Alterar Horário da Loja para Épocas Especiais

Este processo permite ao gerente ajustar o horário de funcionamento da loja em períodos específicos, como festividades, promoções ou eventos especiais. A alteração é feita através da interface de gestão, onde o gerente pode definir novos horários temporários. O sistema regista automaticamente as modificações, garantindo que o novo horário seja considerado na geração dos turnos e comunicando as alterações aos funcionários.

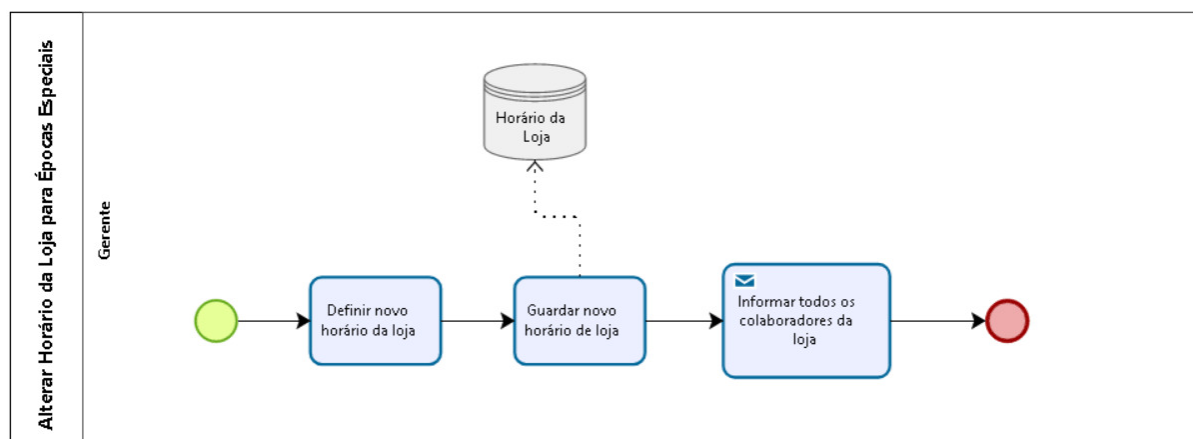


Figura 9: Processo 6 - Alterar Horário da Loja para Épocas Especiais.

2.2.7 Processo 7 - Alterar Dados ou Ativar/Desativar Funcionários

Este processo permite ao gerente atualizar as informações dos funcionários, bem como gerir o seu estado no sistema (ativar ou desativar). A alteração dos dados pode incluir modificações em informações pessoais, funções ou preferências de disponibilidade.

A funcionalidade de **desativação** é utilizada em casos de saída ou despedimento, impedindo o acesso imediato ao sistema e a inclusão do funcionário em escalas futuras. Inversamente, a **reativação** permite recuperar o acesso de colaboradores em casos de readmissão ou regresso de licenças prolongadas, preservando todo o seu histórico de atividade. Todas as modificações são devidamente registadas pelo sistema para garantir a integridade e rastreabilidade das informações dos recursos humanos da loja.

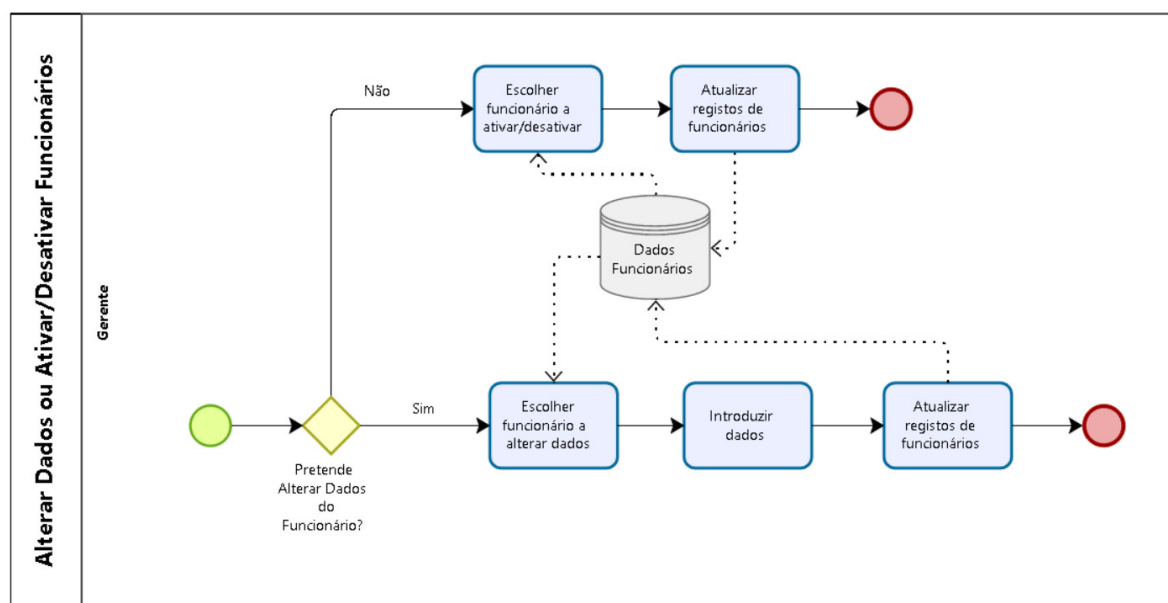


Figura 10: Processo 7 - Alterar Dados ou Ativar/Desativar Funcionários.

3 Levantamento de Requisitos

3.1 Tipos de Utilizador

Esta plataforma conta com três perfis distintos de utilizadores: o **gerente**, o **supervisor** e o **funcionário**.

O **gerente** possui o nível de acesso mais elevado, sendo responsável pela configuração global do sistema. As suas funções incluem a definição de regras de negócio, a gestão de dados dos colaboradores e a geração das escalas de trabalho.

O **supervisor** atua num nível intermédio de gestão operacional. Embora não configure as regras do sistema, possui a responsabilidade crítica de **validar as propostas de horário** geradas pelo gerente, assegurando que estas cumprem os requisitos operacionais da loja antes de serem publicadas para a equipa.

Por fim, o **funcionário** detém um perfil de consulta e interação pessoal. Pode aceder ao seu horário publicado, submeter preferências de disponibilidade (folgas e férias) e iniciar pedidos de troca de turno.

Esta hierarquia de perfis permite que o sistema controle adequadamente os níveis de acesso e as permissões, garantindo que o fluxo de trabalho seja seguro, organizado e eficiente, desde a criação da escala, passando pela validação, até à consulta final.

3.2 Requisitos Funcionais

3.2.1 Requisitos Funcionais de Utilizador

Os requisitos funcionais de utilizador descrevem as ações e funcionalidades que cada tipo de utilizador, como gerente e funcionário, deve poder executar dentro da plataforma de gestão de horários.

Tabela de Requisitos Funcionais de Utilizador

ID	Descrição	Prioridade (MoSCoW)
RFU01	Como Gerente da loja, quero introduzir as informações relativas ao horário de funcionamento da loja para contribuir nas informações necessárias para a criação do horário.	Must
RFU03	Como Gerente da loja, quero poder adicionar funcionários para manter atualizada a informação sobre os recursos humanos da loja.	Must
RFU04	Como Gerente da loja, quero poder alterar os dados dos funcionários registados na loja para caso algum funcionário mude qualquer informação.	Must
RFU05	Como Gerente da loja, quero poder desativar funcionários registados na loja para caso haja algum despedimento ou saída de algum funcionário.	Must
RFU07	Como Gerente da loja, quero ter direito a aprovar as preferências que forem emitidas pelo funcionário no sistema para ter um melhor controlo e estar a par de todas as alterações realizadas na loja.	Must
RFU08	Como Gerente da loja, quero conseguir definir nas regras do sistema a quantidade de funcionários que devem estar a trabalhar simultaneamente no horário de funcionamento da loja para que possa ser possível realizar todas as tarefas necessárias e não haja falta de recursos humanos durante o horário de trabalho.	Must
RFU12	Como Funcionário da loja, quero poder escolher em que dias é que prefiro ter as minhas folgas para que possa ter uma melhor gestão entre vida pessoal e profissional.	Must
RFU14	Como Funcionário da loja, quero ter a oportunidade de, caso surja uma emergência ou por outros motivos de conveniência própria, seja possível fazer a troca de turno com outro colega para uma melhor gestão da vida pessoal com a vida profissional.	Must

ID	Descrição	Prioridade (MoSCoW)
RFU15	Como Supervisor, quero validar (aprovar ou rejeitar) a proposta de horário gerada pelo Gerente para garantir que esta cumpre os requisitos operacionais antes da sua publicação final aos funcionários.	Must
RFU02	Como Gerente da loja, quero poder alterar horário de funcionamento da loja para poder adaptar o horário de funcionamento relativo a épocas especiais como festividades, promoções e eventos.	Should
RFU06	Como Gerente da loja, quero poder verificar se as permutas de horários são possíveis ou não para poder ter um melhor controlo do sistema e estar a par de todas as ações na loja.	Should
RFU09	Como Gerente da loja, quero poder definir o dia em que o horário irá ser lançado para o mês seguinte para que possa ser possível futuras trocas no horário com a devida antecedência caso haja discórdia ou descontentamento em algum turno.	Should
RFU10	Como Funcionário da loja, quero poder escolher as minhas preferências em relação às pessoas com quem trabalho para conseguir trabalhar com as mesmas de modo a aumentar a minha produtividade.	Should
RFU13	Como Funcionário da loja, quero poder escolher em que altura é que prefiro ter as minhas férias para que possa organizar melhor a minha vida pessoal.	Should

3.2.2 Requisitos Funcionais do Sistema

A tabela seguinte apresenta os requisitos funcionais do sistema organizados pela sua prioridade de implementação, segundo o método **MoSCoW**.

Tabela de Requisitos Funcionais do Sistema

ID	Descrição	Prioridade (MoSCoW)
RFS01	O sistema deve garantir igualdade na divisão de turnos entre todos.	Must
RFS02	O sistema deve ter em conta as preferências válidas de cada funcionário na criação do horário.	Must
RFS04	O sistema deve garantir que cada funcionário tenha 2 dias de folga durante a semana.	Must
RFS05	O sistema deve garantir que durante o horário de funcionamento da loja estejam a trabalhar simultaneamente o número mínimo de funcionários definidos pelo gerente.	Must
RFS06	O sistema deve garantir o cumprimento do decreto-lei de 8 horas de descanso entre turnos.	Must
RFS08	O sistema deve lançar o horário do mês seguinte até ao dia definido pelo gerente do mês anterior.	Must
RFS09	O sistema deve gerar o horário de forma a corresponder a todas as informações e regras introduzidas pelos utilizadores.	Must
RFS03	O sistema deve garantir que o gerente e subgerente trabalhem aos sábados, caso definido como regra pelo gerente.	Should
RFS07	O sistema deve garantir que cada funcionário tenha um fim de semana de descanso a cada 7 semanas.	Should

3.3 Requisitos Não Funcionais

Os requisitos não funcionais definem as características de qualidade que o sistema deve cumprir, especificando como o software deve funcionar em termos de desempenho, segurança, usabilidade e outros aspetos técnicos.

Tabela de Requisitos Não Funcionais

ID	Categoria	Descrição
RNF01	Segurança	O acesso ao sistema é realizado através de autenticação por utilizador e palavra-passe.
RNF02	Segurança	As palavras-passe são armazenadas de forma encriptada.
RNF03	Segurança	A sessão expira após 15 minutos de inatividade.
RNF04	Segurança	Apenas o gestor pode aprovar trocas de turnos e definir regras da loja.
RNF05	Segurança	Todas as comunicações entre cliente e servidor devem ser encriptadas (HTTPS).
RNF06	Segurança	O sistema deve cumprir o Regulamento Geral de Proteção de Dados (RGPD), garantindo o consentimento e o direito de acesso ou remoção dos dados pessoais.
RNF07	Segurança	O sistema deve manter um registo (log) das ações importantes, como alterações de horários, aprovações e trocas de turnos, para fins de controlo e auditoria.
RNF06	Desempenho	O sistema deve gerar automaticamente um horário mensal em menos de 10 segundos.
RNF07	Desempenho	O tempo de resposta para qualquer operação (ex.: guardar dados, carregar horários) não deve exceder 3 segundos em condições normais de rede.
RNF08	Operacional	O sistema deve garantir que os dados guardados (como horários, preferências e trocas) não se percam mesmo em caso de falha de energia ou erro de servidor.
RNF09	Operacional	O sistema deve estar disponível pelo menos 99% do tempo, executando períodos programados de manutenção.
RNF10	Operacional	O sistema deve suportar o aumento do número de funcionários e lojas sem degradação significativa do desempenho.
RNF11	Operacional	As configurações devem ser parametrizáveis sem necessidade de reprogramação.
RNF12	Operacional	Deve permitir integração futura com outros módulos (ex: controlo de assiduidade).

ID	Categoria	Descrição
RNF13	Operacional	O sistema deve funcionar nos principais browsers (Chrome, Firefox, Edge).
RNF14	Operacional	O sistema deve ser compatível com diferentes sistemas operativos (Windows, macOS e Android).
RNF15	Operacional	O sistema deve ser compatível com bases de dados relacionais padrão, como MySQL ou PostgreSQL.
RNF06	Manutenção e Suporte	O código do sistema deve ser modular e documentado para permitir futuras alterações e atualizações sem comprometer o funcionamento existente.
RNF06	Manutenção e Suporte	Em caso de falha do sistema, os dados devem poder ser restaurados a partir de cópias de segurança automáticas.
RNF19	Manutenção e Suporte	As atualizações não devem causar perda de dados.
RNF07	Usabilidade	A interface deve ser intuitiva e de fácil utilização.
RNF12	Usabilidade	O sistema deve ser acessível em computador, tablet e smartphone.
RNF17	Usabilidade	As notificações devem ser claras e visíveis.
RNF21	Aparência	A aplicação deve ser responsiva e visualmente coerente.
RNF23	Aparência	Implementação de temas personalizáveis (dark mode, cores customizadas).
RNF05	Legal	O sistema deve garantir conformidade com o decreto-lei (ex: 1 dia de descanso semanal).
RNF22	Cultural e Político	O sistema deve permitir tradução da interface para diferentes idiomas, se necessário.

4 Modelação de Análise do Sistema

A modelação de análise do sistema tem como objetivo representar de forma estruturada e compreensível as funcionalidades que o sistema deverá disponibilizar aos seus utilizadores. Esta fase permite identificar os principais atores envolvidos, as suas interações com o sistema e os diferentes comportamentos esperados, servindo de base para o desenvolvimento e validação dos requisitos definidos anteriormente.

4.1 Modelo de Casos de Uso

O modelo de casos de uso descreve as interações entre os utilizadores (atores) e o sistema, especificando as funcionalidades que cada um pode executar.

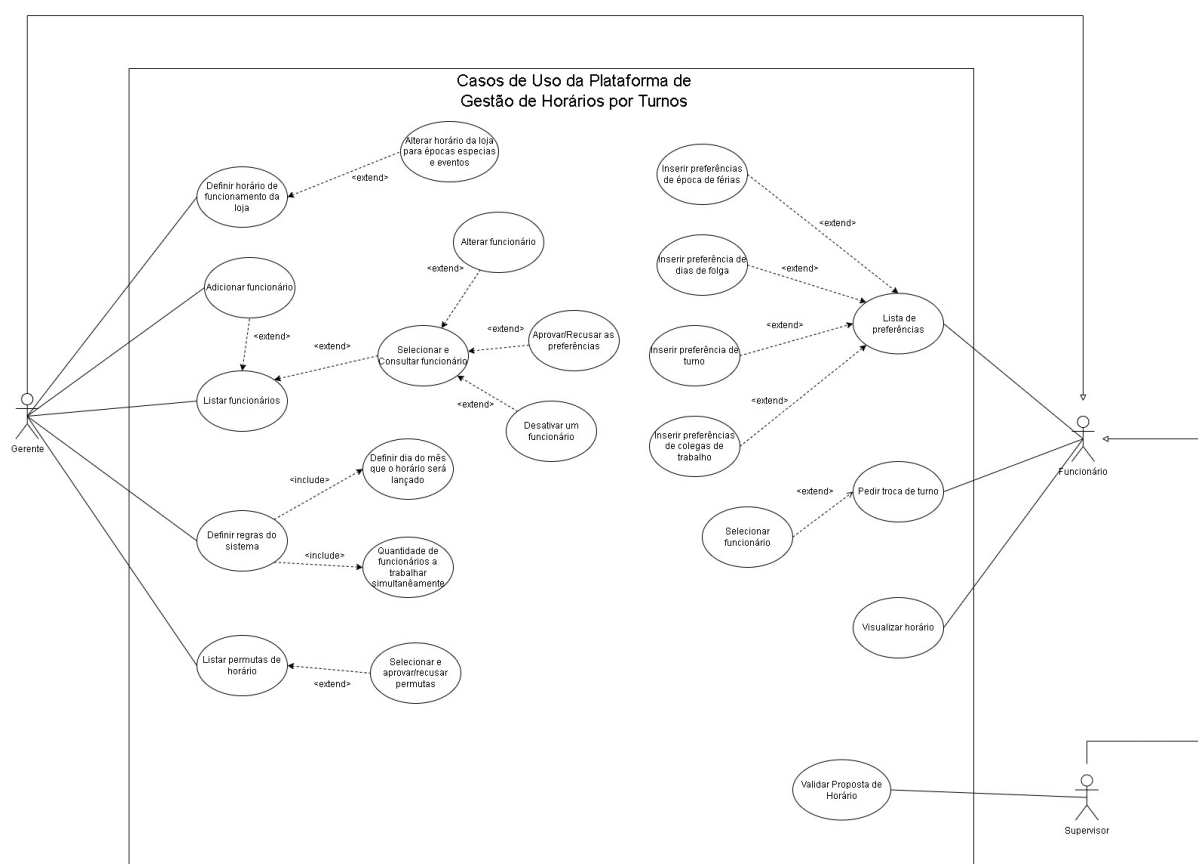


Figura 11: *Modelo de Casos de Uso.*

4.2 Modelo de classes do Domínio

O modelo de classes de domínio representa as entidades principais do sistema e os seus relacionamentos, capturando a estrutura lógica e conceptual da gestão de horários, turnos, utilizadores, lojas e processos associados (permutas, pedidos de folga, etc.).

As principais entidades identificadas incluem:

- **Utilizador:** representa os funcionários, gerentes e subgerentes do sistema.
- **Cargos e Tipos de Cargo:** definem os papéis hierárquicos dos utilizadores (gerente, supervisor, funcionário).
- **Loja e LojaUtilizador:** gerem as lojas/filiais e a associação de utilizadores a cada loja.
- **Turno e Horário:** representam os turnos de trabalho e os horários atribuídos.
- **Permuta e DayOff:** modelam os pedidos de troca de turnos e de ausência (férias/folgas).
- **Regras e RegrasLoja:** permitem definir e personalizar regras de negócio por loja.

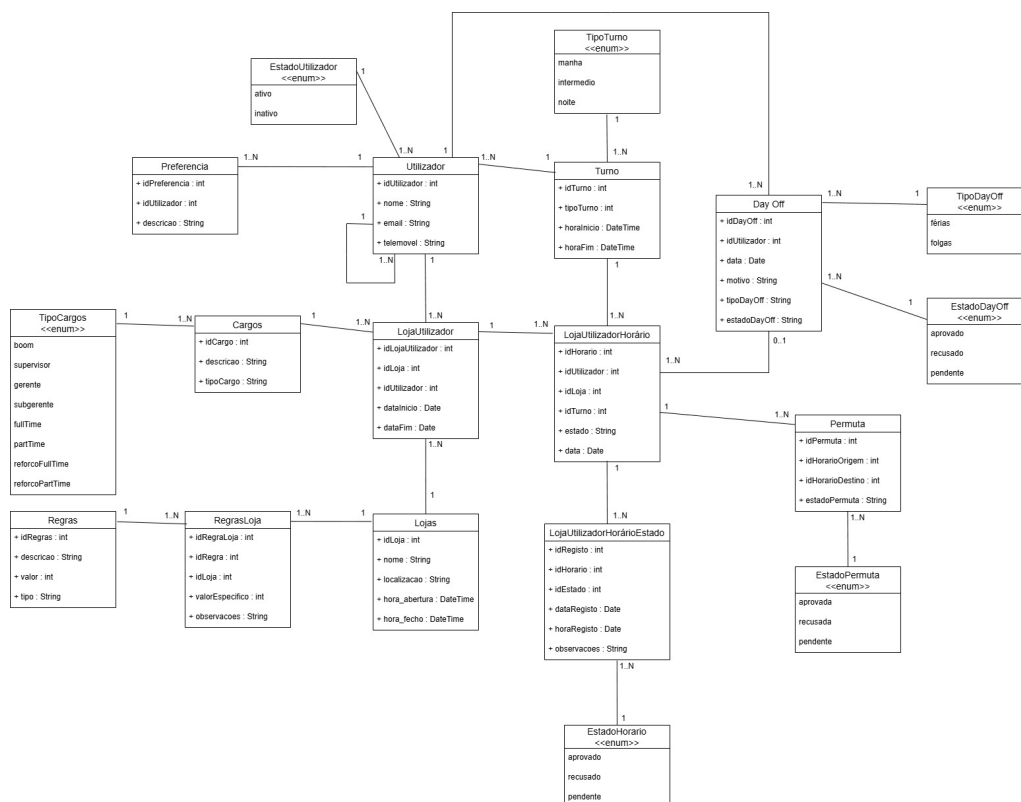


Figura 12: *Modelo de Classes de Domínio.*

4.3 Diagramas de transição de estados

Os diagramas de transição de estados (DTE) descrevem o ciclo de vida das entidades do sistema, especificando os estados possíveis e as transições permitidas entre eles.

4.3.1 Diagrama de Transição de Estados - Utilizador

O DTE do Utilizador descreve os estados relacionados com o registo e gestão da conta de um utilizador no sistema. Um utilizador que está **Registado** pode estar **Ativo** ou **Inativo**.

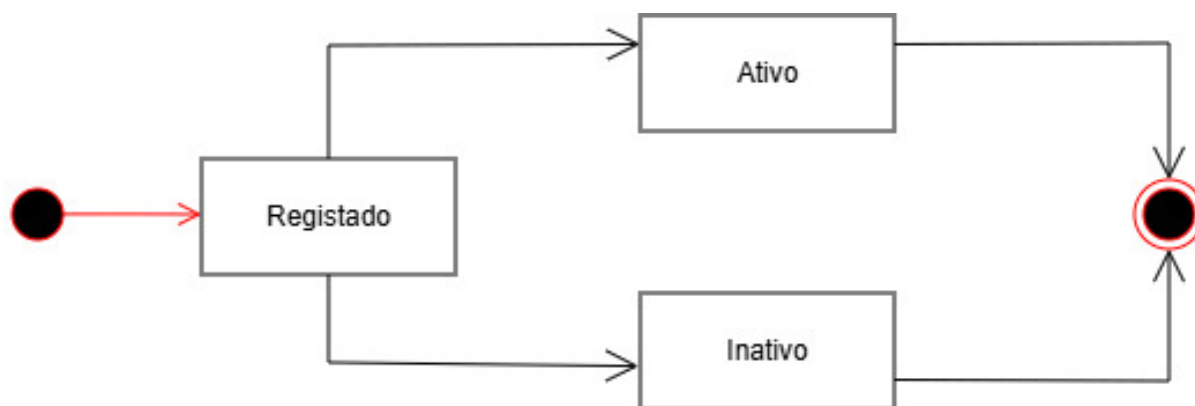


Figura 13: *Diagrama de Transição de Estados - Utilizador.*

4.3.2 Diagrama de Transição de Estados - Horário

O DTE do Horário representa o ciclo de vida de uma escala gerada. Inicialmente, o horário encontra-se no estado **Gerado**. De seguida, passa para **Pendente**, aguardando a validação operacional. O estado **Sugestão** pode ocorrer caso existam propostas de alteração pontuais antes da aprovação final.

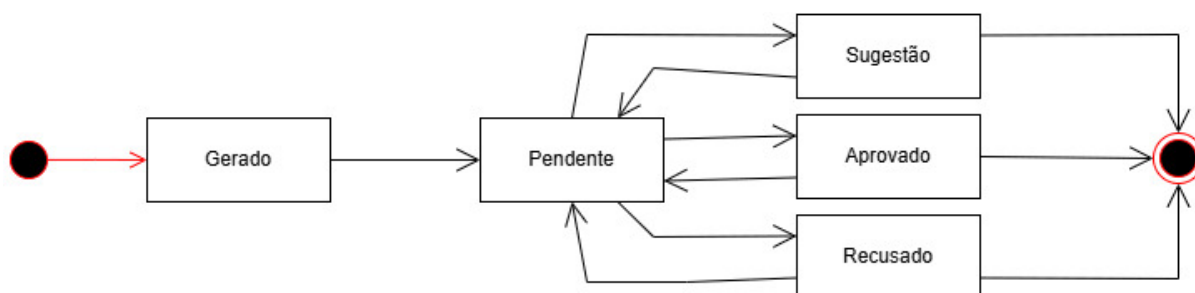


Figura 14: *Diagrama de Transição de Estados - Horário.*

4.3.3 Diagrama de Transição de Estados - Permuta

O DTE da Permuta descreve o fluxo de um pedido de troca de turnos entre dois funcionários. O processo inicia-se com o estado **Pedida**, passando para **Pendente** após submissão.

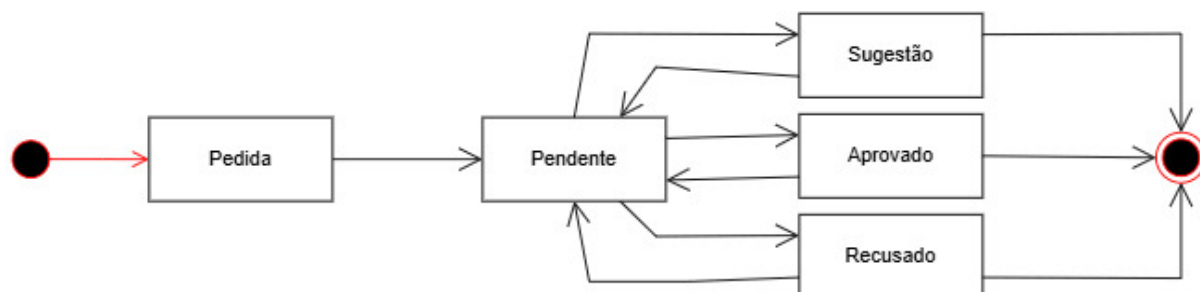


Figura 15: *Diagrama de Transição de Estados - Permuta.*

4.3.4 Diagrama de Transição de Estados - DayOff

O DTE do DayOff modela o ciclo de um pedido de folga ou férias. O pedido pode ser **Pendente** de análise, **Aprovado** ou **Recusado** pelo gerente.

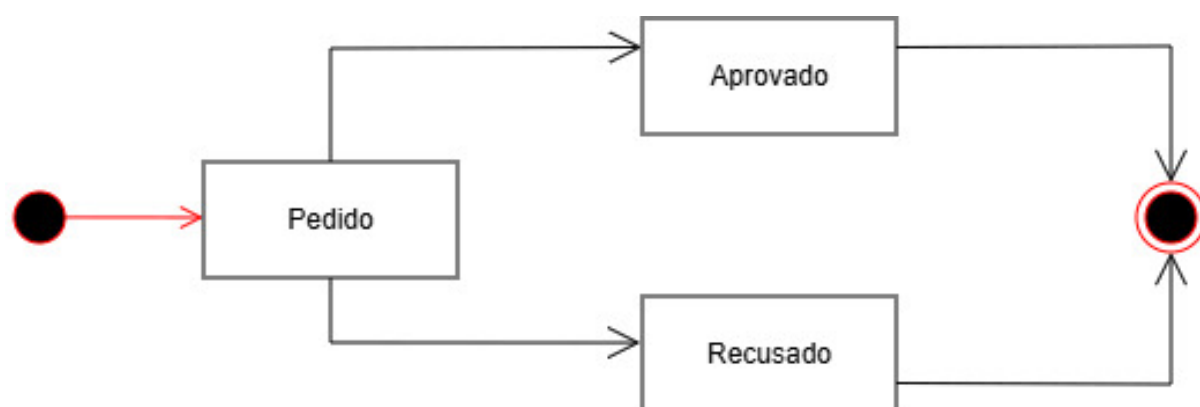


Figura 16: *Diagrama de Transição de Estados - DayOff.*

4.4 Especificação Detalhada de Casos de Uso

4.4.1 Especificação usando Template de Casos de Uso

4.4.1.1 Caso de Uso: Definir horário de funcionamento da loja

ID: UC-G01	Caso de Uso: Definir horário de funcionamento da loja
Requisitos:	RFU01
Ator Principal:	Gerente
Pré-condições:	Gerente autenticado no sistema
Pós-condições:	Horário base de funcionamento da loja definido e registado
Cenário Principal:	1. Gerente acede a "Definir Horário de Funcionamento" 2. Sistema mostra horário atual (se existir) 3. Gerente define: - Dias da semana de funcionamento - Horário de abertura e fecho por dia - Turnos padrão (manhã, tarde, noite) 4. Sistema valida consistência 5. Sistema guarda horário como base para geração de escalas 6. Sistema confirma "Horário definido com sucesso"
Cenários Alternativos:	CA1: Horário já definido 1. Sistema mostra opção "Manter atual" ou "Redefinir" 2. Gerente escolhe redefinir e segue fluxo principal
Exceções	E1: Tentativa de definir horário impossível (ex: 25 horas por dia) E2: Conflito com feriados ou dias de encerramento obrigatório

Tabela 4: Caso de Uso UC-G01: Definir horário de funcionamento da loja

4.4.1.2 Caso de Uso: Alterar horário da loja para épocas especiais e eventos

ID: UC-G01a	Caso de Uso: Alterar horário da loja para épocas especiais e eventos
Tipo:	Extensão de "Definir horário de funcionamento da loja"
Requisitos:	RFU02
Ator Principal:	Gerente
Pré-condições:	Horário base já definido (RFU01)
Pós-condições:	Horário especial registado para período definido
Cenário Principal:	1. Gerente acede a "Horários Especiais" 2. Seleciona período (ex: Natal, Promoção) 3. Define horário excecional 4. Sistema aplica automaticamente às escalas 5. Sistema notifica funcionários
Cenários Alternativos:	CA1: Período especial sobrepõe-se a outro já definido 1. Sistema alerta para sobreposição 2. Gerente pode: a) Cancelar nova definição b) Substituir período existente
Exceções	E1: Período especial ultrapassa limites legais (ex: mais de 12h/dia) E2: Falha na notificação dos funcionários

Tabela 5: Caso de Uso UC-G01a: Alterar horário para épocas especiais

4.4.1.3 Caso de Uso: Alterar funcionário

ID: UC-G03a	Caso de Uso: Alterar funcionário
Tipo:	Extensão de "Selecionar e Consultar funcionário"
Requisitos:	RFU04
Ator Principal:	Gerente
Pré-condições:	Funcionário existente no sistema
Pós-condições:	Dados do funcionário atualizados
Cenário Principal:	1. Gerente seleciona funcionário a alterar 2. Sistema abre formulário de edição 3. Gerente modifica dados necessários 4. Sistema valida alterações 5. Sistema atualiza e regista em histórico
Cenários Alternativos:	CA1: Alteração de cargo com impacto em horários futuros 1. Sistema alerta para impacto nas escalas 2. Gerente confirma alteração e regenera horários afetados
Exceções	E1: Tentativa de alterar funcionário desativado E2: Conflito de dados com outros registos

Tabela 6: Caso de Uso UC-G03a: Alterar funcionário

4.4.1.4 Caso de Uso: Desativar um funcionário

ID: UC-G03b	Caso de Uso: Desativar um funcionário
Tipo:	Extensão de "Selecionar e Consultar funcionário"
Requisitos:	RFU05
Ator Principal:	Gerente
Pré-condições:	Funcionário ativo no sistema
Pós-condições:	Funcionário marcado como "Inativo"
Cenário Principal:	1. Gerente seleciona "Desativar Funcionário" 2. Sistema pede confirmação e motivo 3. Sistema altera estado para "Inativo" 4. Sistema remove de escalas futuras 5. Sistema bloqueia login
Cenários Alternativos:	CA1: Funcionário com escalas futuras atribuídas 1. Sistema alerta para escalas afetadas 2. Gerente pode: a) Cancelar desativação b) Prosseguir e reassignar turnos
Exceções	E1: Tentativa de desativar único gerente ativo E2: Funcionário já desativado anteriormente

Tabela 7: Caso de Uso UC-G03b: Desativar funcionário

4.4.1.5 Caso de Uso: Definir regras do sistema

ID: UC-G04	Caso de Uso: Definir regras do sistema
Requisitos:	RFU08 , RFU09
Ator Principal:	Gerente
Pré-condições:	Gerente autenticado
Pós-condições:	Regras do sistema definidas e ativas
Cenário Principal:	1. Gerente acede a "Definir Regras do Sistema" 2. Define regras incluindo: - Quantidade funcionários/turno («include») - Dia de lançamento horário («include») 3. Sistema valida consistência 4. Sistema ativa regras
Cenários Alternativos:	CA1: Regras conflituosas com legislação 1. Sistema alerta para inconformidade legal 2. Gerente ajusta regras para cumprir legislação
Exceções	E1: Tentativa de definir regras ilegais E2: Conflito entre novas regras e horários já gerados

Tabela 8: Caso de Uso UC-G04: Definir regras do sistema

4.4.1.6 Caso de Uso: Listar pedidos permutas de horário

ID: UC-G05	Caso de Uso: Listar pedidos permutas de horário
Requisitos:	RFU06
Ator Principal:	Gerente
Pré-condições:	Pedidos de permuta submetidos
Pós-condições:	Lista de permutas apresentada
Cenário Principal:	1. Gerente acede a "Permutas Pendentes" 2. Sistema mostra tabela com análise 3. Gerente pode filtrar e ordenar
Cenários Alternativos:	CA1: Nenhuma permuta pendente 1. Sistema mostra mensagem "Sem permutas pendentes" 2. Oferece opção para ver histórico ou estatísticas
Exceções	E1: Erro ao carregar dados das permutas E2: Permuta expirada durante a consulta

Tabela 9: Caso de Uso UC-G05: Listar pedidos permutas

4.4.1.7 Caso de Uso: Lista de preferências

ID: UC-F01	Caso de Uso: Lista de preferências
Requisitos:	RFU12, RFU13, RFU10
Ator Principal:	Funcionário
Pré-condições:	Funcionário autenticado
Pós-condições:	Interface de preferências apresentada
Cenário Principal:	1. Funcionário acede a "Minhas Preferências" 2. Sistema mostra categorias: - Dias de folga («extend»RFU12) - Férias («extend»RFU13) - Colegas («extend»RFU10) - Turnos («extend») 3. Visualiza estado das suas preferências
Cenários Alternativos:	CA1: Nenhuma preferência definida 1. Sistema mostra mensagem "Sem preferências definidas" 2. Oferece botão "Adicionar Primeira Preferência"
Exceções	E1: Funcionário desativado (acesso bloqueado) E2: Erro ao carregar preferências do servidor

Tabela 10: Caso de Uso UC-F01: Lista de preferências

4.4.1.8 Caso de Uso: Pedir troca de turno

ID: UC-F02	Caso de Uso: Pedir troca de turno
Requisitos:	RFU14
Ator Principal:	Funcionário
Pré-condições:	Horário publicado, turno atribuído
Pós-condições:	Pedido de troca criado
Cenário Principal:	1. Funcionário acede a "Pedir Troca de Turno" 2. Seleciona turno para trocar 3. Sistema lista colegas («extend») 4. Escolhe colega 5. Sistema valida viabilidade 6. Sistema cria pedido e notifica
Cenários Alternativos:	CA1: Nenhum colega disponível para troca 1. Sistema sugere "Lista de espera para troca" 2. Funcionário pode entrar na lista de espera
Exceções	E1: Turno já iniciado (não pode ser trocado) E2: Colega já tem permuta pendente com outro funcionário

Tabela 11: Caso de Uso UC-F02: Pedir troca de turno

4.4.1.9 Caso de Uso: Validar Proposta de Horário

ID: UC-S01	Caso de Uso: Validar Proposta de Horário
Requisitos:	RFU15
Ator Principal:	Supervisor
Pré-condições:	Supervisor autenticado e existência de um horário em estado "Pendente"
Pós-condições:	Estado do horário atualizado ("Publicado" ou "Rejeitado")
Cenário Principal:	1. Supervisor recebe notificação de nova proposta de horário 2. Accede ao menu "Validação de Horários" 3. Sistema apresenta a visualização da escala proposta 4. Supervisor analisa distribuição e cumprimento de regras 5. Supervisor aprova a proposta 6. Sistema altera estado para "Publicado" 7. Sistema envia notificações automáticas aos funcionários
Cenários Alternativos:	CA1: Supervisor rejeita a proposta 1. Supervisor identifica problemas na escala 2. Seleciona a opção "Rejeitar/Devolver" 3. Sistema solicita motivo obrigatório 4. Supervisor insere observações para o Gerente 5. Sistema altera estado para "Correção Necessária" e notifica o Gerente
Exceções	E1: Horário removido pelo Gerente durante a análise E2: Falha no envio das notificações de publicação

Tabela 12: Caso de Uso UC-S01: Validar Proposta de Horário

4.4.2 Especificação usando Diagramas de atividades

Esta secção apresenta os diagramas de atividades para os casos de uso mais complexos do sistema, ilustrando os fluxos de trabalho e decisões envolvidas em cada processo.

4.4.2.1 Diagrama de Atividades: Gerar Horário Automático (UC-S01)

Descrição do fluxo:

- O processo inicia com a verificação da data de lançamento programada (RFS08)
- São carregadas todas as regras ativas, funcionários ativos e preferências aprovadas
- O algoritmo de geração é executado, considerando igualdade na distribuição (RFS01) e preferências válidas (RFS02)
- O horário gerado é validado contra todas as regras do sistema (RFS09)
- Após validação bem-sucedida, o horário é publicado e notificados todos os funcionários
- Em caso de conflito, o gerente é notificado para intervenção

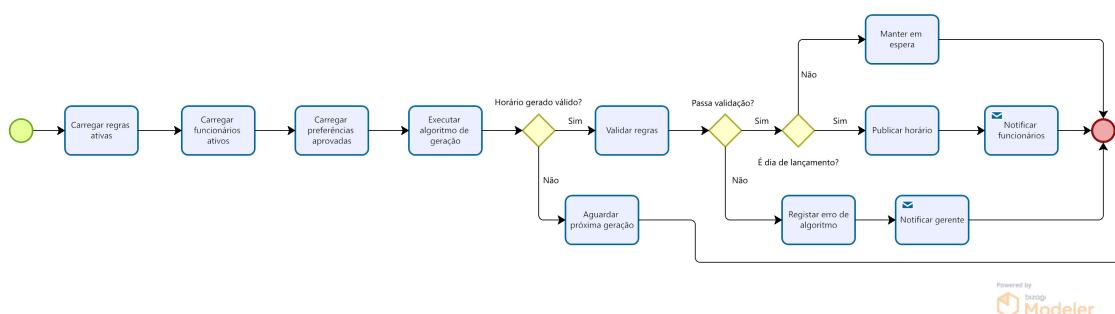


Figura 17: Diagrama de atividades para o caso de uso *Gerar Horário Automático*. Mostra o processo de geração automática do horário mensal, incluindo validações das regras do sistema (RFS01-09) e notificações aos funcionários.

4.4.2.2 Diagrama de Atividades: Selecionar e aprovar/recusar permutas (UC-G05a)

Descrição do fluxo:

- O gerente acede à lista de permutas pendentes
- Cada permuta é analisada quanto à sua viabilidade (RFU06)
- O gerente pode: Aprovar, Recusar (com motivo obrigatório) ou Pedir mais informação
- Se aprovada, os horários são atualizados e ambos os funcionários notificados
- Todas as decisões são registadas em log de auditoria

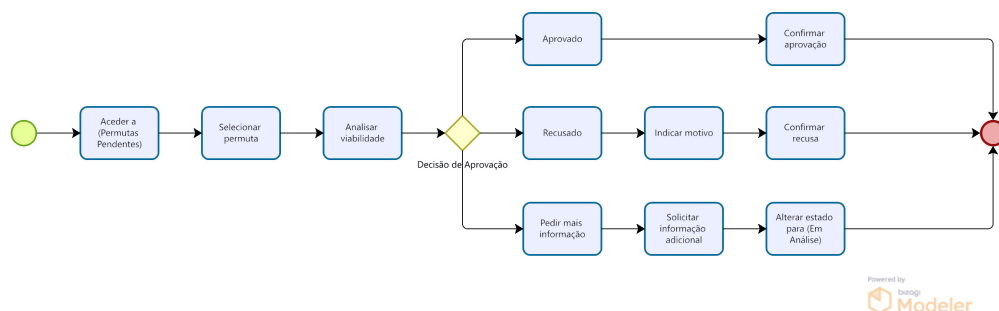


Figura 18: Diagrama de atividades para o caso de uso *Selecionar e aprovar/recusar permutas*. Ilustra o processo de análise e decisão do gerente sobre pedidos de troca de turnos entre funcionários, incluindo a verificação de viabilidade (RFU06).

4.4.2.3 Diagrama de Atividades: Inserir preferência de dias de folga (UC-F01a)

Descrição do fluxo:

- O funcionário seleciona os dias preferidos para folga
- O sistema valida se a seleção está dentro dos limites permitidos (RFS04)
- Verifica-se conflitos com outras preferências já existentes
- Se válido, a preferência é criada com estado "Pendente" para aprovação do gerente (RFU07)
- O funcionário recebe confirmação da submissão

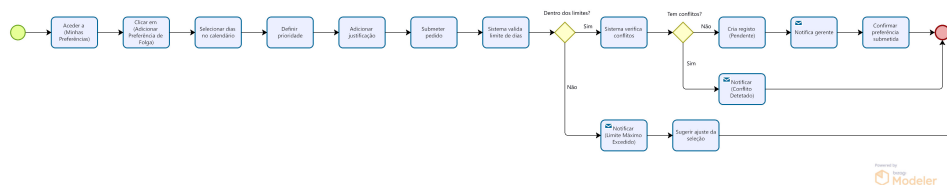


Figura 19: Diagrama de atividades para o caso de uso *Inserir preferência de dias de folga*. Mostra o processo de um funcionário submeter os seus dias preferidos para folga, incluindo validações de limites (RFS04) e aprovação pelo gerente (RFU07).

4.4.2.4 Diagrama de Atividades: Inserir preferências de época de férias (UC-F01d)

Descrição do fluxo:

- O funcionário seleciona o período preferido para férias
- O sistema valida: período dentro do ano, dias disponíveis, e ausência de sobreposições
- Se todas as validações forem bem-sucedidas, a preferência é submetida para aprovação
- Caso contrário, o funcionário é notificado do erro específico para correção

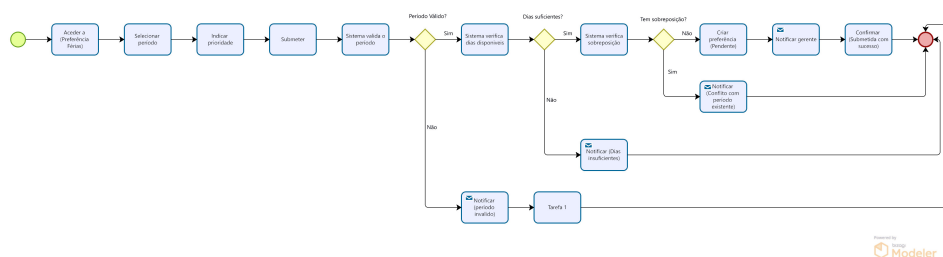


Figura 20: Diagrama de atividades para o caso de uso *Inserir preferências de época de férias*. Ilustra o processo de submissão de períodos preferidos para férias, incluindo validações de disponibilidade e conflitos com outros pedidos.

4.4.2.5 Diagrama de Atividades: Aprovar/Recusar as preferências (UC-G03c)

Descrição do fluxo:

- O gerente analisa cada preferência pendente
- Considera o impacto no horário futuro e o equilíbrio da equipa
- Toma uma decisão: Aprovar, Recusar (com motivo) ou Sugerir alterações
- O funcionário é notificado da decisão
- Preferências aprovadas são consideradas na próxima geração de horário

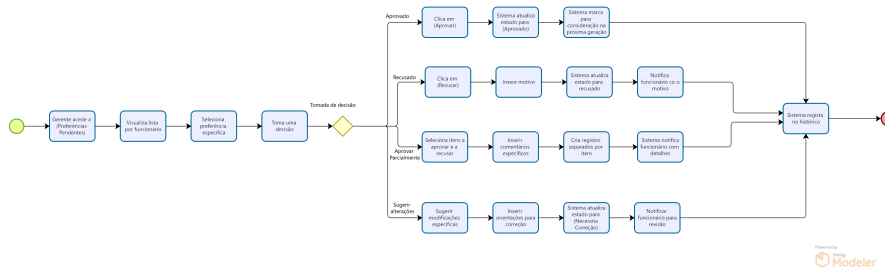


Figura 21: Diagrama de atividades para o caso de uso *Aprovar/Recusar as preferências*. Mostra o processo de análise e decisão do gerente sobre as preferências submetidas pelos funcionários (RFU07).

4.4.2.7 Diagrama de Atividades: Inserir preferências de colegas de trabalho (UC-F01c)

Descrição do fluxo:

- O funcionário acede à secção de preferências de colegas
- Visualiza a lista de colegas ativos disponíveis
- Seleciona os colegas com quem prefere trabalhar
- (Opcional) Identifica colegas que prefere evitar
- Define o nível de preferência para cada seleção
- O sistema valida se todos os colegas selecionados ainda estão ativos
- Verifica conflitos (ex: mesmo colega em listas opostas)
- Se todas as validações forem bem-sucedidas, a preferência é submetida para aprovação do gerente
- O funcionário recebe confirmação da submissão

Requisitos relacionados:

- **RFU10** (Should): Como Funcionário, quero poder escolher as minhas preferências em relação às pessoas com quem trabalho

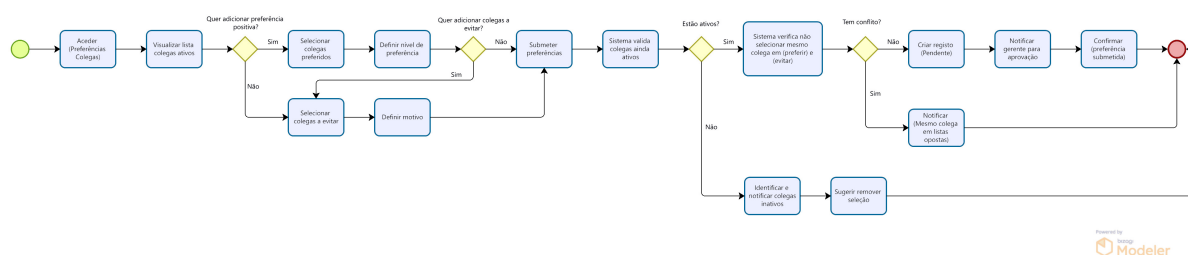


Figura 23: Diagrama de atividades para o caso de uso *Inserir preferências de colegas de trabalho*. Mostra o processo de um funcionário definir os colegas com quem prefere trabalhar, incluindo validações de disponibilidade e aprovação (RFU10).

4.4.2.8 Diagrama de Atividades: Definir dia do mês que o horário será lançado (UC-G04b)

Descrição do fluxo:

- O gerente acede à configuração do calendário de publicação
- Seleciona o dia do mês para publicação do horário (RFU09)
- Define a hora específica para a publicação automática
- O sistema valida se a data selecionada é um dia útil
- Em caso de data inválida (fim-de-semana), sugere o dia útil mais próximo
- Após confirmação, o sistema agenda a publicação automática (RFS08)
- São configuradas notificações prévias para alertar o gerente
- A configuração é guardada e aplicada para os meses seguintes

Requisitos relacionados:

- **RFU09** (Should): Como Gerente, quero poder definir o dia em que o horário irá ser lançado
- **RFS08** (Must): O sistema deve lançar o horário do mês seguinte até ao dia definido pelo gerente

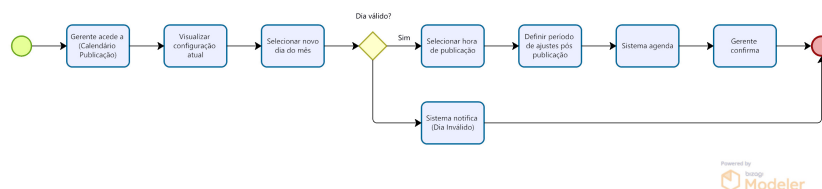


Figura 24: Diagrama de atividades para o caso de uso *Definir dia do mês que o horário será lançado*. Mostra o processo de configuração da data de publicação automática do horário mensal (RFU09, RFS08).

4.4.3 Especificação usando Diagramas de sequência

Nesta secção são apresentados os diagramas de sequência que modelam a interação entre os atores e o sistema para os casos de uso seleccionados. Estes diagramas permitem visualizar o fluxo de mensagens e a ordem temporal das operações, complementando a descrição textual dos casos de uso.

4.4.3.1 Caso de Uso: Visualizar Horário (UC-F03)

O diagrama de sequência correspondente ao caso de uso **UC-F03 – Visualizar Horário** encontra-se representado na Figura 25. Este diagrama detalha a sequência de interações entre o Funcionário, o Sistema e a Base de Dados, conforme descrito no cenário principal e nos respetivos cenários alternativos e exceções.

Explicação do Fluxo:

1. **Cenário Principal:** O funcionário, já autenticado, acede à opção "Meu Horário". O sistema consulta a base de dados para obter o horário do período atual. Se este estiver publicado e disponível, é apresentada uma visão mensal ou semanal com os detalhes de cada turno. O funcionário tem ainda a opção de exportar o horário.
2. **Cenário Alternativo (CA1):** Se o horário para o período em questão ainda não tiver sido publicado, o sistema informa o utilizador com a mensagem "Horário em preparação" e oferece automaticamente a visualização do horário do mês anterior.
3. **Exceções:** São ainda tratados dois cenários excepcionais: (E1) um erro técnico ao carregar os dados e (E2) a situação em que o funcionário não tem turnos atribuídos para o período solicitado, sendo apresentada uma mensagem informativa adequada em cada caso.

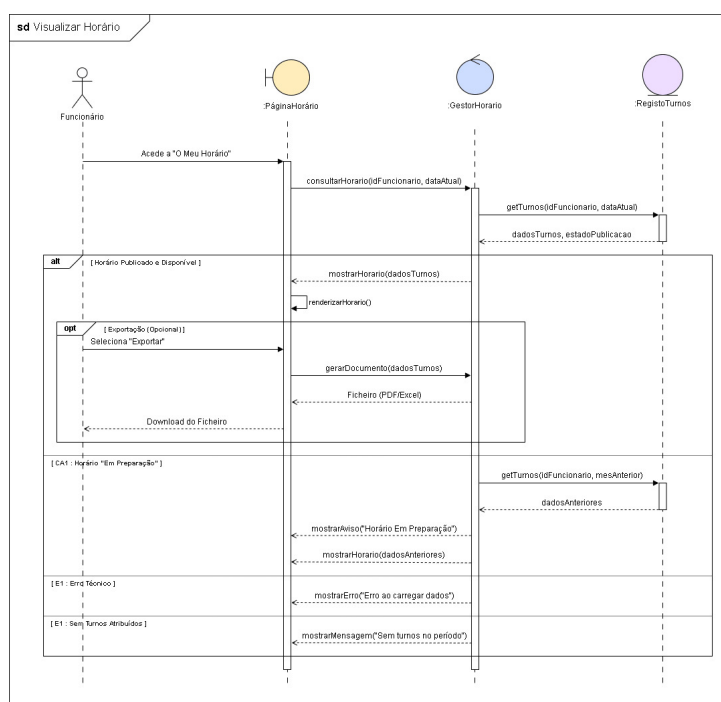


Figura 25: Diagrama de sequência do caso de uso UC-F03: Visualizar Horário

4.4.3.2 Caso de Uso: Adicionar Funcionário (UC-G02a)

O diagrama de sequência correspondente ao caso de uso **UC-G02a – Adicionar Funcionário**, uma extensão de "Listar Funcionários", é apresentado na Figura 26. Este diagrama ilustra o processo realizado por um Gerente para registar um novo colaborador no sistema.

Explicação do Fluxo:

1. **Cenário Principal:** Após autenticação, o gerente inicia o processo a partir da lista de funcionários. O sistema apresenta um formulário para preenchimento dos dados pessoais e profissionais do novo funcionário. Antes do registo, o sistema valida a unicidade do email fornecido. Se válido, cria o perfil com o estado "Ativo" e envia automaticamente um email com as credenciais de acesso para o novo utilizador.
2. **Cenário Alternativo (CA1):** Caso o email inserido já esteja registado no sistema, é apresentado um alerta ao gerente. Este deve, então, solicitar um email alternativo ao funcionário (processo externo ao sistema) e retomar o preenchimento do formulário com o novo email, seguindo o fluxo normal de registo.
3. **Exceções:** O diagrama também modela a gestão de exceções, nomeadamente: (E1) uma falha no envio do email com as credenciais, situação em que o funcionário é criado mas o gerente é notificado da falha; e (E2) a submissão do formulário com dados obrigatórios em falta, que resulta numa mensagem de erro e na necessidade de completar os campos em falta.

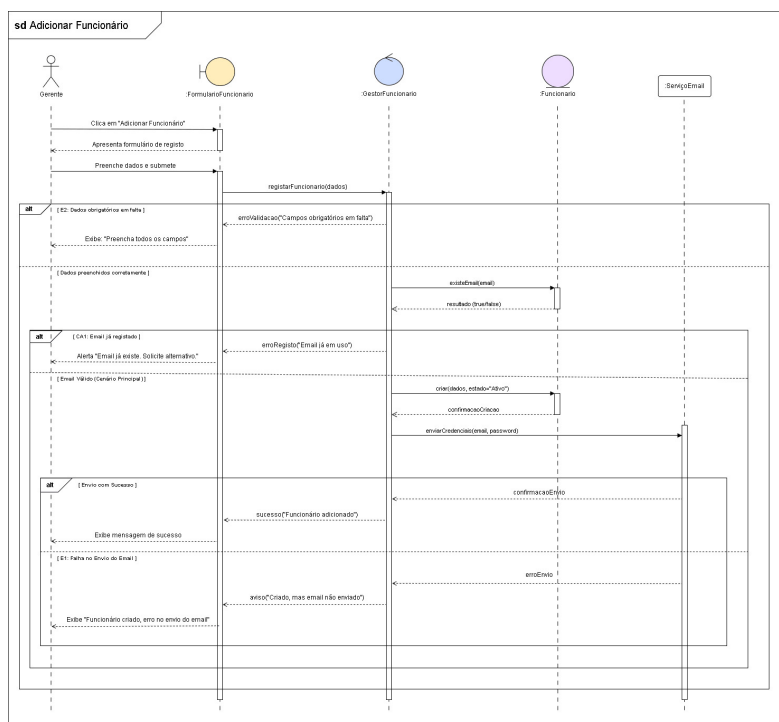


Figura 26: Diagrama de sequência do caso de uso UC-G02a: Adicionar Funcionário

5 Desenho e Implementação da BD

5.1 Modelação de Dados

A modelação de dados do sistema de gestão de horários tem como objetivo identificar e estruturar a informação necessária para suportar os processos de negócio da loja Levi's Braga Parque. Esta modelação foi realizada com base nos requisitos funcionais definidos, nos processos BPMN elaborados e nas regras de negócio estabelecidas pelo gestor.

O sistema necessita de armazenar informação relativa aos funcionários, horários, turnos, preferências, regras da loja e permutas de horários, garantindo integridade, consistência e flexibilidade dos dados.

As principais entidades identificadas na modelação de dados são as seguintes:

- **Loja:** representa a unidade comercial onde o sistema é aplicado, armazenando informações como o horário de funcionamento e parâmetros gerais definidos pelo gerente.
- **Funcionário:** representa cada colaborador da loja, abrangendo todos os perfis hierárquicos (**gerente, supervisor e equipa de loja**), contendo os seus dados pessoais, cargo e estado (ativo/inativo).
- **Turno:** define os diferentes períodos de trabalho padrão (ex: manhã, tarde, noite), com as respetivas horas de início e fim.
- **Horário/Escala:** representa a atribuição concreta de um turno a um funcionário numa data específica.
- **Preferência:** armazena os pedidos dos funcionários relativamente a folgas, férias e colegas de trabalho, sujeitos à aprovação hierárquica.
- **Permuta:** representa os pedidos de troca de turno entre funcionários, incluindo o ciclo de vida do pedido (pendente, aprovada ou rejeitada).
- **Regra:** contém as parametrizações do sistema (regras de negócio e legais) configuráveis pelo gerente para a geração automática.

A modelação foi realizada de acordo com princípios de normalização, evitando redundância de dados e assegurando que cada entidade possui uma responsabilidade bem definida. Esta estrutura permite que o sistema seja facilmente adaptável a outras empresas (multi-loja), bastando ajustar as regras e parâmetros.

5.1.1 Estrutura de tabelas inicial (Pré-Normalização)

Numa primeira abordagem (correspondente ao Diagrama de Entidade-Relacionamento inicial), o sistema foi idealizado com uma estrutura simplificada que, embora funcional para casos básicos, apresentava limitações de integridade e redundância de dados. As tabelas definidas inicialmente eram:

- **Loja:** (id_loja, nome, localizacao, hora_abertura, hora_fecho)
- **Utilizador:** (id_utilizador, nome, email, password, nome_cargo, **id_loja**)

(Nota: A inclusão do id_loja aqui limitava o funcionário a trabalhar em apenas um local, impedindo a rotação entre lojas e criando dependência rígida. O cargo como texto "nome_cargo" gerava redundância.)

- **Turno:** (id_turno, designacao, hora_inicio, hora_fim)
- **Regras:** (id_regra, descricao, valor, **id_loja**)

(Nota: A inclusão do id_loja diretamente na regra obrigava a repetir a definição da mesma regra várias vezes para lojas diferentes, dificultando a manutenção global.)

- **Horario:** (id_horario, data, estado, **id_utilizador**, **id_turno**)
- **Preferencias:** (id_preferencia, descricao, **id_utilizador**)
- **Permuta:** (id_permuta, data_pedido, estado, **id_horario_origem**, **id_horario_destino**)

Legenda: Chave Primária (PK) | **Chave Estrangeira (FK)**

5.1.2 Estrutura de tabelas final (Pós-Normalização)

Após a aplicação das formas normais (1FN, 2FN e 3FN), a estrutura foi refinada para eliminar redundâncias e suportar relacionamentos complexos, como a possibilidade de um funcionário ter histórico em várias lojas. Destaca-se a criação da tabela associativa LojaUtilizador e a separação de Cargos e Regras_Loja.

- **Lojas:** (id_loja, nome, localizacao, hora_abertura, hora_fecho)
- **Cargos:** (id_cargo, nome, tipo, descricao)
- **Utilizadores:** (id_utilizador, nome, email, telemovel, password_hash, estado)
- **Regras:** (id_regra, descricao, valor_padrao, tipo)
- **Turnos:** (id_turno, tipo, hora_inicio, hora_fim)
- **LojaUtilizador:** (id_lojautilizador, **id_utilizador**, **id_loja**, **id_cargo**, data_inicio, data_fim)
- **Preferencias:** (id_preferencia, **id_utilizador**, descricao)
- **Regras_Loja:** (id_regra_loja, **id_loja**, **id_regra**, valor_especifico, observacoes)
- **Day_Offs:** (id_dayoff, **id_utilizador**, data_ausencia, motivo, tipo, estado)
- **Horarios:** (id_horario, **id_lojautilizador**, **id_turno**, data_turno, estado)
- **Permutas:** (id_permuta, **id_horario_origem**, **id_horario_destino**, estado, data_pedido)
- **Historico_Horario_Estados:** (id_registo, **id_horario**, estado_novo, data_registo, observacoes)

Legenda: Chave Primária (PK) | **Chave Estrangeira (FK)**

5.2 Modelo Físico da Base de dados

O modelo físico da base de dados representa a implementação técnica da estrutura de informação definida anteriormente. Este modelo traduz as entidades e relações para tabelas, colunas e restrições específicas do Sistema de Gestão de Base de Dados (SGBD) escolhido.

A Figura 27 apresenta o Diagrama de Entidade-Relacionamento (DER) desenvolvido, detalhando as chaves primárias (PK), chaves estrangeiras (FK) e os tipos de dados atribuídos a cada campo, assegurando a integridade referencial do sistema.

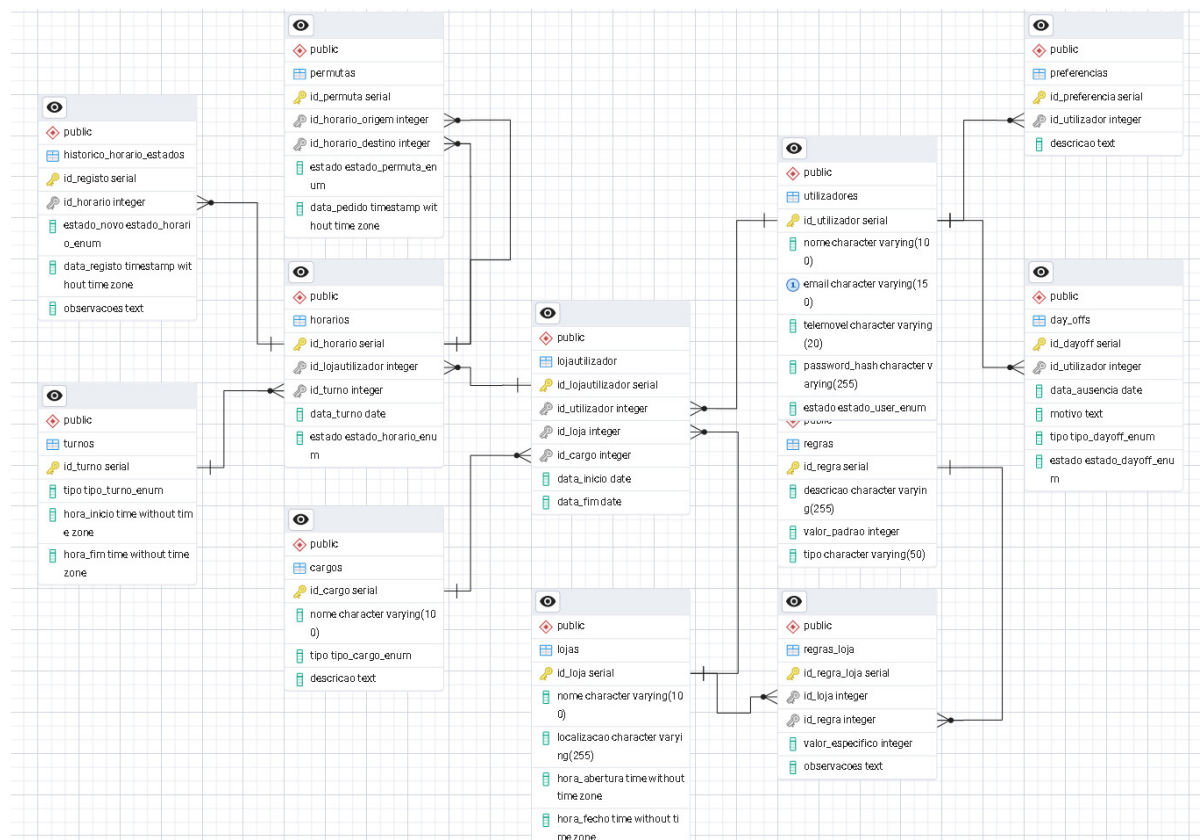


Figura 27: Modelo Físico da Base de Dados (DER).

5.3 Tecnologias usadas e Criação da Base de Dados

A implementação física da base de dados foi realizada utilizando o Sistema de Gestão de Base de Dados (SGBD) **PostgreSQL**, gerido através da interface gráfica **pgAdmin 4**. A escolha do PostgreSQL deve-se à sua robustez, conformidade com os padrões SQL e suporte avançado para integridade referencial. Para garantir a consistência dos dados, foram utilizados tipos enumerados (**ENUM**) e restrições de chave estrangeira. Abaixo apresenta-se o script SQL desenvolvido, formatado para execução direta no pgAdmin 4.

1. Definição de Tipos e Tabelas Independentes

O script inicia-se com a criação de tipos de dados personalizados (ENUMs) para padronizar os estados e categorias, seguido pelas tabelas mestre.

```

1  -- 1. Definição dos Tipos (ENUMs)
2  CREATE TYPE tipo_cargo_enum AS ENUM ('boom', 'supervisor', 'gerente',
3    'subgerente', 'fulltime', 'parttime', 'reforco_fulltime', '
4    reforco_parttime');
5  CREATE TYPE estado_user_enum AS ENUM ('ativo', 'inativo');
6  CREATE TYPE tipo_turno_enum AS ENUM ('manha', 'intermedio', 'noite');
7  CREATE TYPE tipo_dayoff_enum AS ENUM ('ferias', 'folgas', 'baixa');
8  CREATE TYPE estado_dayoff_enum AS ENUM ('aprovado', 'recusado', '
9    pendente');
10 CREATE TYPE estado_permuta_enum AS ENUM ('aprovada', 'recusada', '
11    pendente');
12 CREATE TYPE estado_horario_enum AS ENUM ('aprovado', 'recusado', '
13    pendente');
14
15 -- 2. Tabelas Independentes
16 CREATE TABLE lojas (
17   id_loja SERIAL PRIMARY KEY,
18   nome VARCHAR(100) NOT NULL,
19   localizacao VARCHAR(255),
20   hora_abertura TIME NOT NULL,
21   hora_fecho TIME NOT NULL
22 );
23
24 CREATE TABLE cargos (
25   id_cargo SERIAL PRIMARY KEY,
26   nome VARCHAR(100),
27   tipo tipo_cargo_enum NOT NULL,
28   descricao TEXT
29 );
30
31 CREATE TABLE utilizadores (
32   id_utilizador SERIAL PRIMARY KEY,
33   nome VARCHAR(100) NOT NULL,
34   email VARCHAR(150) UNIQUE NOT NULL,
35   telemovel VARCHAR(20),
36   password_hash VARCHAR(255) NOT NULL,
37   estado estado_user_enum DEFAULT 'ativo'
38 );
39
40 CREATE TABLE regras (
41   id_regra SERIAL PRIMARY KEY,
42   descricao VARCHAR(255) NOT NULL,
43   valor_padrao INT,
44   tipo VARCHAR(50)
45 );
46
47 CREATE TABLE turnos (
48   id_turno SERIAL PRIMARY KEY,
49   tipo tipo_turno_enum NOT NULL,
50   hora_inicio TIME NOT NULL,
51   hora_fim TIME NOT NULL
52 );

```

Código 1: Criação de Tipos e Tabelas Principais

2. Tabelas de Associação e Dependentes

Nesta secção, estabelecem-se as relações entre as entidades através de chaves estrangeiras.

```

1  -- 3. Tabela de Associação
2  CREATE TABLE lojautilizador (
3      id_lojautilizador SERIAL PRIMARY KEY,
4      id_utilizador INT NOT NULL,
5      id_loja INT NOT NULL,
6      id_cargo INT NOT NULL,
7      data_inicio DATE NOT NULL DEFAULT CURRENT_DATE,
8      data_fim DATE,
9      CONSTRAINT fk_utilizador FOREIGN KEY (id_utilizador)
10         REFERENCES utilizadores(id_utilizador),
11      CONSTRAINT fk_loja FOREIGN KEY (id_loja) REFERENCES lojas(id_loja),
12      CONSTRAINT fk_cargo FOREIGN KEY (id_cargo) REFERENCES cargos(
13         id_cargo)
14 );
15
16 -- 4. Tabelas Dependentes
17 CREATE TABLE preferencias (
18     id_preferencia SERIAL PRIMARY KEY,
19     id_utilizador INT NOT NULL,
20     descricao TEXT NOT NULL,
21     CONSTRAINT fk_pref_user FOREIGN KEY (id_utilizador)
22         REFERENCES utilizadores(id_utilizador)
23 );
24
25 CREATE TABLE regras_loja (
26     id_regra_loja SERIAL PRIMARY KEY,
27     id_loja INT NOT NULL,
28     id_regra INT NOT NULL,
29     valor_especifico INT,
30     observacoes TEXT,
31     CONSTRAINT fk_regra_loja FOREIGN KEY (id_loja) REFERENCES lojas(
32         id_loja),
33     CONSTRAINT fk_regra_def FOREIGN KEY (id_regra) REFERENCES regras(
34         id_regra)
35 );
36
37 CREATE TABLE day_offs (
38     id_dayoff SERIAL PRIMARY KEY,
39     id_utilizador INT NOT NULL,
40     data_ausencia DATE NOT NULL,
41     motivo TEXT,
42     tipo tipo_dayoff_enum NOT NULL,
43     estado estado_dayoff_enum DEFAULT 'pendente',
44     CONSTRAINT fk_dayoff_user FOREIGN KEY (id_utilizador)
45         REFERENCES utilizadores(id_utilizador)
46 );
47
48 CREATE TABLE horarios (
49     id_horario SERIAL PRIMARY KEY,
50     id_lojautilizador INT NOT NULL,
51     id_turno INT NOT NULL,
52     data_turno DATE NOT NULL,
53     estado estado_horario_enum DEFAULT 'pendente',
54     CONSTRAINT fk_horario_lojautilizador FOREIGN KEY (id_lojautilizador
55 
```

```

52     REFERENCES lojautilizador(id_lojautilizador),
53     CONSTRAINT fk_horario_turno FOREIGN KEY (id_turno)
54     REFERENCES turnos(id_turno)
55 );
56
57 CREATE TABLE permutas (
58     id_permuta SERIAL PRIMARY KEY,
59     id_horario_origem INT NOT NULL,
60     id_horario_destino INT NOT NULL,
61     estado estado_permuta_enum DEFAULT 'pendente',
62     data_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
63     CONSTRAINT fk_permuta_origem FOREIGN KEY (id_horario_origem)
64     REFERENCES horarios(id_horario),
65     CONSTRAINT fk_permuta_destino FOREIGN KEY (id_horario_destino)
66     REFERENCES horarios(id_horario)
67 );
68
69 CREATE TABLE historico_horario_estados (
70     id_registo SERIAL PRIMARY KEY,
71     id_horario INT NOT NULL,
72     estado_novo estado_horario_enum NOT NULL,
73     data_registo TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
74     observacoes TEXT,
75     CONSTRAINT fk_hist_horario FOREIGN KEY (id_horario)
76     REFERENCES horarios(id_horario)
77 );

```

Código 2: Tabelas de Associação e Dependentes

3. Otimização

Por fim, a criação de índices para otimização de performance.

```

1  -- 5. Índices
2  CREATE INDEX idx_lojautilizador_user ON lojautilizador(id_utilizador);
3  CREATE INDEX idx_lojautilizador_loja ON lojautilizador(id_loja);
4  CREATE INDEX idx_horario_data ON horarios(data_turno);
5  CREATE INDEX idx_horario_estado ON horarios(estado);

```

Código 3: Índices de Performance

5.4 Queries à Base de Dados

Para validar a estrutura da base de dados e suportar as funcionalidades principais do sistema, foram desenvolvidas diversas consultas SQL (Queries). Abaixo, apresentam-se os exemplos mais relevantes que suportam os requisitos de visualização de horários, gestão de equipas e controlo de trocas.

1. Consultar o Horário Mensal de um Funcionário

Esta consulta permite obter a lista de turnos de um funcionário específico para um determinado mês, cruzando dados das tabelas de utilizadores, contratos, horários e turnos. Suporta o requisito funcional de visualização do próprio horário.

```

1 SELECT
2     u.nome AS funcionario,
3     h.data_turno,
4     t.tipo AS turno,
5     t.hora_inicio,
6     t.hora_fim,
7     h.estado
8 FROM horarios h
9 JOIN lojautilizador lu ON h.id_lojautilizador = lu.id_lojautilizador
10 JOIN utilizadores u ON lu.id_utilizador = u.id_utilizador
11 JOIN turnos t ON h.id_turno = t.id_turno
12 WHERE u.email = 'francisco.gomes@levi.com'
13     AND h.data_turno BETWEEN '2025-05-01' AND '2025-05-31'
14 ORDER BY h.data_turno ASC;
```

Código 4: Query para obter o horário mensal de um funcionário

2. Verificar Disponibilidade da Equipa (Quem trabalha hoje?)

Utilizada pelo Gerente para verificar a escala diária da loja, garantindo que o número mínimo de funcionários (RFS05) está a ser cumprido.

```

1 SELECT
2     l.nome AS loja,
3     u.nome AS funcionario,
4     c.nome AS cargo,
5     t.hora_inicio,
6     t.hora_fim
7 FROM horarios h
8 JOIN lojautilizador lu ON h.id_lojautilizador = lu.id_lojautilizador
9 JOIN utilizadores u ON lu.id_utilizador = u.id_utilizador
10 JOIN cargos c ON lu.id_cargo = c.id_cargo
11 JOIN lojas l ON lu.id_loja = l.id_loja
12 JOIN turnos t ON h.id_turno = t.id_turno
13 WHERE h.data_turno = '2025-05-15' -- Data a consultar
14     AND h.estado = 'aprovado'
15 ORDER BY t.hora_inicio ASC;
```

Código 5: Query para listar funcionários escalados numa data específica

3. Análise de Equidade na Distribuição de Turnos

Esta consulta de agregação é fundamental para cumprir o requisito de igualdade na divisão de turnos (RFS01), permitindo ao sistema ou ao gerente visualizar quantos turnos cada funcionário realizou no mês.

```

1 SELECT
2     u.nome ,
3     COUNT(h.id_horario) AS total_turnos ,
4     SUM(EXTRACT(HOUR FROM (t.hora_fim - t.hora_inicio))) AS total_horas
5 FROM horarios h
6 JOIN lojautilizador lu ON h.id_lojautilizador = lu.id_lojautilizador
7 JOIN utilizadores u ON lu.id_utilizador = u.id_utilizador
8 JOIN turnos t ON h.id_turno = t.id_turno
9 WHERE h.data_turno BETWEEN '2025-05-01' AND '2025-05-31'
10 GROUP BY u.nome
11 ORDER BY total_turnos DESC;

```

Código 6: Contagem de turnos por funcionário para controlo de equidade

4. Listar Pedidos de Permuta Pendentes

Consulta utilizada no painel de gestão do Gerente para identificar trocas de turno que necessitam de aprovação. Envolve a junção da tabela de permutas com a informação dos dois horários envolvidos (origem e destino).

```

1 SELECT
2     p.id_permuta ,
3     u1.nome AS solicitante ,
4     h1.data_turno AS data_origem ,
5     u2.nome AS substituto ,
6     h2.data_turno AS data_destino ,
7     p.data_pedido
8 FROM permutas p
9 JOIN horarios h1 ON p.id_horario_origem = h1.id_horario
10 JOIN lojautilizador lu1 ON h1.id_lojautilizador = lu1.id_lojautilizador
11 JOIN utilizadores u1 ON lu1.id_utilizador = u1.id_utilizador
12 JOIN horarios h2 ON p.id_horario_destino = h2.id_horario
13 JOIN lojautilizador lu2 ON h2.id_lojautilizador = lu2.id_lojautilizador
14 JOIN utilizadores u2 ON lu2.id_utilizador = u2.id_utilizador
15 WHERE p.estado = 'pendente'
16 ORDER BY p.data_pedido ASC;

```

Código 7: Listagem de permutas pendentes de aprovação

5. Validação de Horários (Perfil Supervisor)

Esta consulta permite ao Supervisor listar apenas os horários que foram gerados mas que ainda aguardam validação (estado 'pendente'), suportando o caso de uso UC-S01.

```
1 SELECT
2     h.id_horario ,
3     u.nome AS funcionario ,
4     c.nome AS cargo ,
5     h.data_turno ,
6     t.tipo AS turno ,
7     t.hora_inicio ,
8     t.hora_fim
9 FROM horarios h
10 JOIN lojautilizador lu ON h.id_lojautilizador = lu.id_lojautilizador
11 JOIN utilizadores u ON lu.id_utilizador = u.id_utilizador
12 JOIN cargos c ON lu.id_cargo = c.id_cargo
13 JOIN turnos t ON h.id_turno = t.id_turno
14 WHERE h.estado = 'pendente'
15 ORDER BY h.data_turno ASC;
```

Código 8: Query para Supervisor listar horários pendentes de aprovação

6 Arquitetura e Implementação do Sistema (Projeto II)

6.1 Arquitetura em Camadas

A implementação do sistema foi realizada com base numa arquitetura em camadas, suportada por **Java**, **Spring Boot**, **Spring Data JPA**, **PostgreSQL** e **JavaFX**. A aplicação encontra-se organizada em:

- **Views JavaFX / FXML**: responsáveis pela interface gráfica;
- **Controllers**: recebem eventos da interface e coordenam os fluxos;
- **BLL**: concentram as regras de negócio e validações;
- **Repositories**: suportam o acesso a dados com Spring Data JPA;
- **Base de Dados PostgreSQL**: persiste utilizadores, lojas, horários, folgas, permutas, preferências, auditoria e propostas mensais.

A integração entre JavaFX e Spring Boot permitiu criar controladores como *beans* do Spring, simplificando a injeção de dependências e a separação entre interface e lógica de negócio.

6.2 Mapeamento Objeto-Relacional (JPA/Hibernate)

O acesso à base de dados foi implementado com **JPA** e **Hibernate**. Esta tecnologia permitiu converter automaticamente os registos da base de dados em objetos Java, facilitando a manipulação de entidades e relações.

Por exemplo, a entidade **Horario** encontra-se associada a **Turno** e **LojaUtilizador**, permitindo ao sistema construir a informação necessária para dashboards, relatórios, validação do supervisor e gestão de permutas.

6.3 Camada de Lógica de Negócio (BLL)

A **BLL** representa o núcleo funcional da aplicação. Em vez de a interface interagir diretamente com a base de dados, os pedidos passam por classes de serviço que aplicam regras e validações.

6.3.1 Exemplo de validação de autenticação

```
1 public Utilizador autenticar(String email, String password) {  
2     Utilizador utilizador = utilizadorRepository.findByEmailIgnoreCase(  
3         email.trim());  
4  
5     if (utilizador == null) {  
6         return null;  
7     }  
8  
9     if (!passwordEncoder.matches(password, utilizador.getPasswordHash()  
10    )) {  
11         return null;  
12     }  
13  
14     if (!utilizador.estaAtivo()) {  
15         return null;  
16     }  
17  
18     return utilizador;  
19 }
```

Código 9: Exemplo simplificado de autenticação e validação de utilizador ativo

6.3.2 Principais fluxos implementados

Na entrega desktop foram concretizados os seguintes módulos:

- autenticação e controlo de sessão;
- dashboard principal com navegação por perfil;
- gestão da loja e das regras operacionais;
- gestão de funcionários;
- preferências do funcionário e aprovação pela gerência;
- pedidos de folga com histórico e decisão;
- permutas com validação e histórico;
- geração de propostas mensais de horário;
- validação do horário pelo supervisor;
- relatórios mensais de horas;
- perfil do utilizador;
- auditoria de eventos relevantes.

6.4 Segurança e auditoria

A aplicação implementa autenticação por utilizador e palavra-passe, controlo de utilizadores ativos, hashing de palavras-passe com BCrypt e registo de eventos relevantes de sessão, login e operações críticas. Foi ainda previsto um controlo de expiração de sessão por inatividade, reforçando a segurança operacional da aplicação desktop.

7 Implementação da Interface Desktop (Fase 2)

7.1 Transição para Interface Gráfica

A interface desktop constitui a principal materialização desta entrega. A aplicação foi desenvolvida em **JavaFX** e integrada com o ecossistema **Spring Boot**, permitindo articular interface gráfica, regras de negócio e persistência de dados de forma coesa.

A camada de apresentação foi desenhada para servir diferentes perfis de utilizador, disponibilizando menus, permissões e fluxos de trabalho específicos para colaboradores, supervisores e gerência.

7.2 Tecnologias de Prototipagem e Design

Antes da codificação das janelas e controladores (utilizando frameworks de interface gráfica Java), recorreu-se à criação de *mockups* de alta fidelidade para garantir o alinhamento com os casos de uso previamente definidos. Para o efeito, utilizou-se a ferramenta de Inteligência Artificial generativa **Google Stitch**, que permitiu transformar requisitos técnicos e de negócio em interfaces visuais modernas. O design seguiu um estilo caracterizado por uma paleta de cores de alto contraste (Branco e Preto) pontuada pelo icónico vermelho da marca (*Levi's*).

7.3 Mockups e Protótipos de Ecrã

Os protótipos desenvolvidos refletem os principais fluxos de interação dos diferentes perfis de utilizador da loja.

7.3.1 Ecrã de Autenticação (Login)

O ecrã de entrada foi desenhado com um layout centrado e visualmente alinhado com a identidade do projeto. Integra os campos de autenticação, feedback de erro, controlo de visibilidade da palavra-passe e acesso seguro ao dashboard principal após validação.

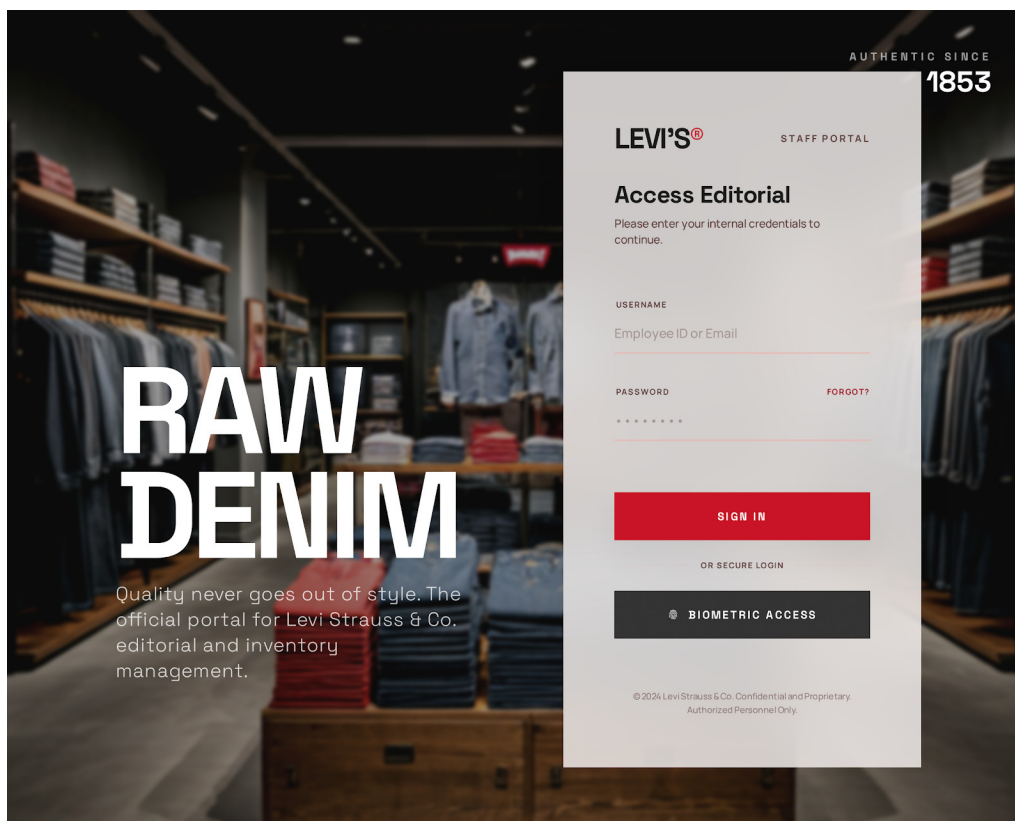


Figura 28: Protótipo de Alta Fidelidade - Ecrã de Login da Aplicação Desktop.

7.3.2 Painel do Gerente (*Manager Dashboard*)

O ecrã dedicado ao perfil de gestão foca-se na eficiência operacional e visão global da loja. A interface apresenta um painel central de resumo com alertas imediatos (como pedidos de permuta pendentes de validação) e botões de ação rápida para processos críticos, destacando-se a funcionalidade de geração automática das escalas mensais.

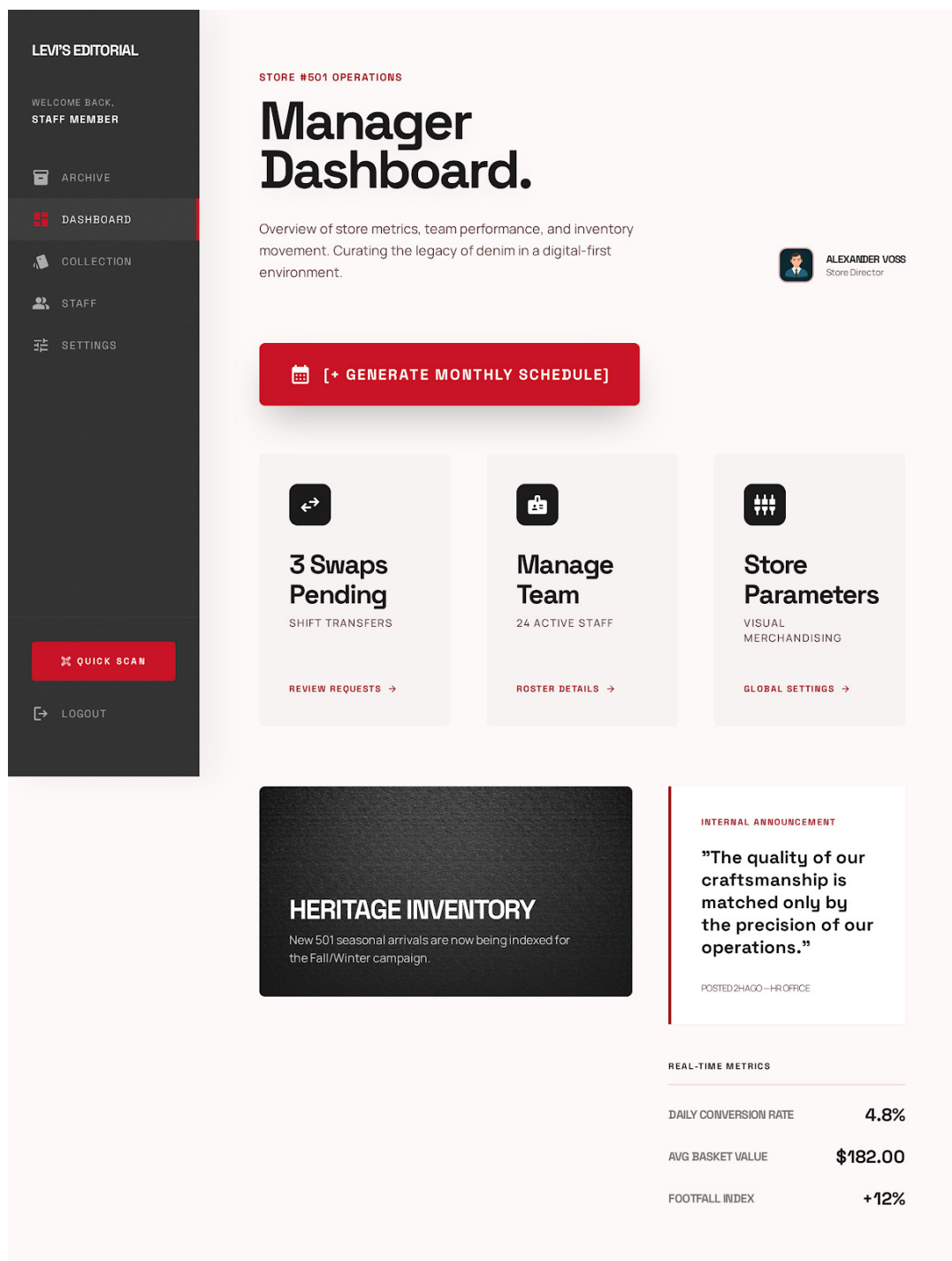


Figura 29: Protótipo de Alta Fidelidade - Painel do Gerente (Dashboard Operacional).

7.3.3 Painel do Funcionário (*Employee Dashboard*)

A área de trabalho do funcionário privilegia a clareza da informação individual. A secção central exibe os próximos turnos do colaborador logado, organizados de forma estruturada (Data, Turno, Estado). Na secção inferior ("Management Suite"), foram integrados atalhos interativos que permitem acionar rapidamente às ações mais frequentes: consulta do horário, pedido de folga, permuta, preferências e edição do perfil.

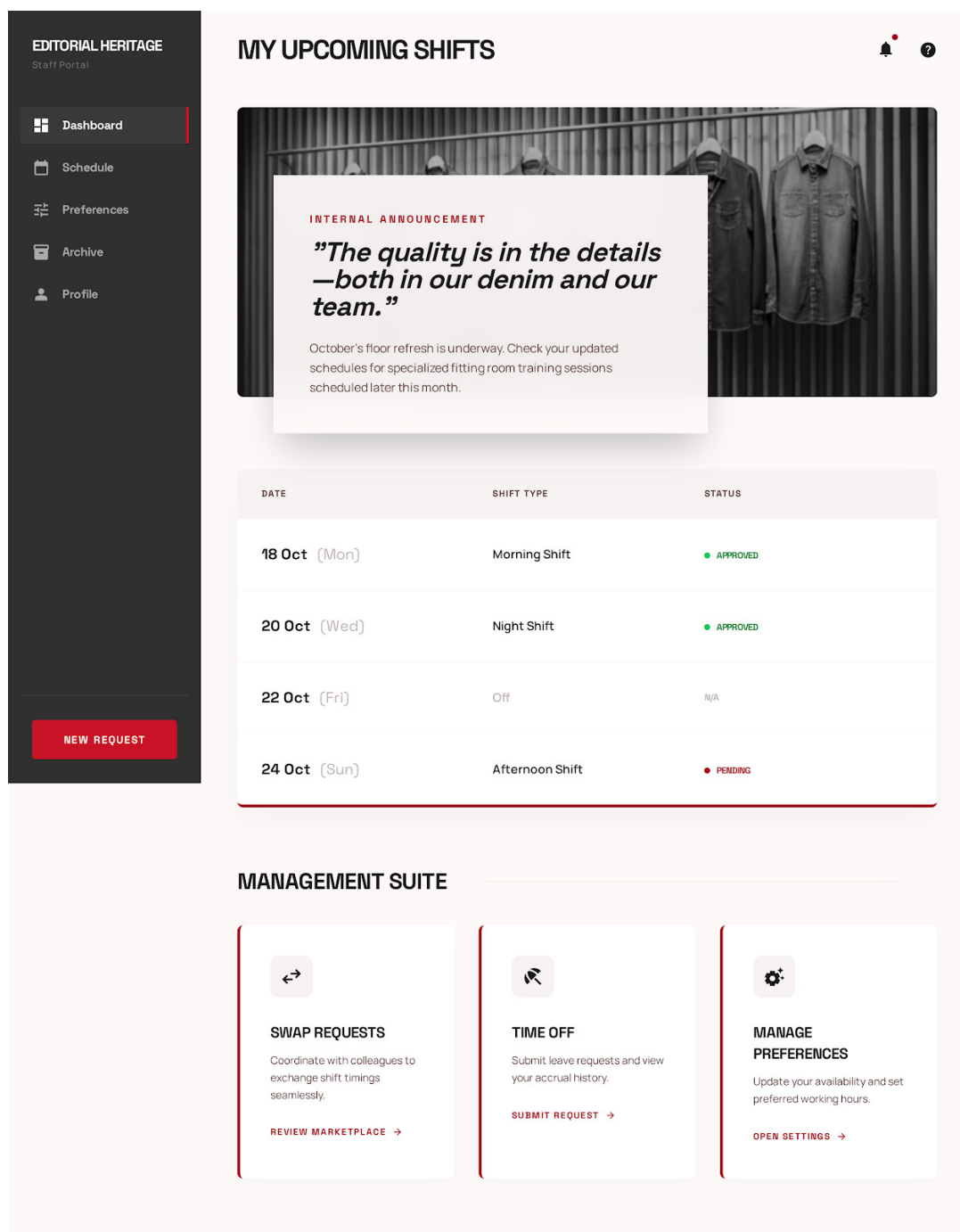


Figura 30: Protótipo de Alta Fidelidade - Painel do Funcionário com listagem de turnos e atalhos de gestão.

7.4 Implementação funcional da aplicação desktop

A aplicação atual disponibiliza uma navegação lateral e superior coerente com o perfil autenticado. Os fluxos principais foram distribuídos por ecrãs próprios, recorrendo a controladores JavaFX especializados e à camada BLL para suportar as ações do utilizador.

Entre os aspetos implementados destacam-se:

- dashboards diferenciados por perfil;
- visualização de próximos turnos;
- navegação para gestão da loja, funcionários, horários, preferências, folgas, permutas, auditoria e relatórios;
- mensagens de validação e confirmação antes de operações críticas;
- modais personalizados com consistência visual;
- suporte a execução em ambiente desktop para demonstração.

7.5 Integração entre interface, lógica e dados

Após autenticação, o sistema preserva o contexto do utilizador autenticado e utiliza esse estado para determinar permissões e conteúdos visíveis. Os controllers invocam a BLL, que consulta os repositórios JPA e atualiza a interface com dados reais provenientes do PostgreSQL.

Esta abordagem permitiu construir uma aplicação funcional de ponta a ponta, desde o login até à consulta, decisão e persistência das principais operações do sistema.

8 Implementação da Plataforma Web (Fase 3)

8.1 Interface do Funcionário – Módulo Complementares

A componente web da plataforma expõe os fluxos de funcionário, supervisor e gestão através de controladores Spring MVC. O `WebComplementaresController` centraliza os endpoints responsáveis pela submissão e aprovação de folgas, preferências e permutas, todos acessíveis a partir do caminho base `/web/complementares` e partilhando integralmente a mesma BLL da aplicação desktop.

Endpoints do `WebComplementaresController` (base: `/web/complementares`)

Método HTTP	Rota	Parâmetros	Objetivo
GET	/	<code>origemPermuta*</code>	Carregar a página com os pedidos do utilizador e as filas de aprovação
POST	<code>/folgas</code>	<code>dataAusencia</code> , <code>tipo</code> , <code>motivo*</code>	Submeter um pedido de folga
POST	<code>/folgas/{idDayOff}/aprovar</code>	<code>{idDayOff}</code> (<i>path</i>)	Aprovar um pedido de folga pendente
POST	<code>/folgas/{idDayOff}/rejeitar</code>	<code>{idDayOff}</code> (<i>path</i>)	Rejeitar um pedido de folga pendente
POST	<code>/preferencias</code>	<code>tipo</code> , <code>descricao</code> , <code>dataInicio*</code> , <code>dataFim*</code> , <code>prioridade*</code>	Registar uma nova preferência de horário
POST	<code>/preferencias/{idPreferencia}/aprovar</code>	<code>{idPreferencia}</code> (<i>path</i>), <code>decisao*</code>	Aprovar a preferência de um funcionário
POST	<code>/preferencias/{idPreferencia}/rejeitar</code>	<code>{idPreferencia}</code> (<i>path</i>), <code>decisao*</code>	Rejeitar a preferência de um funcionário
POST	<code>/preferencias/{idPreferencia}/remover</code>	<code>{idPreferencia}</code> (<i>path</i>)	Remover uma preferência própria
POST	<code>/permutas</code>	<code>idHorarioOrigem</code> , <code>idHorarioDestino</code>	Submeter um pedido de troca de turno

(continua na página seguinte)

(continuação da tabela 13)

Método HTTP	Rota	Parâmetros	Objetivo
POST	/permutas/{idPermuta}/aprovar	{idPermuta} (path)	Aprovar um pedido de permuta de turno
POST	/permutas/{idPermuta}/rejeitar	{idPermuta} (path)	Rejeitar um pedido de permuta de turno
POST	/permutas-folga/{idPermutaFolga}/aprovar	{idPermutaFolga} (path)	Aprovar uma permuta de folga entre funcionários
POST	/permutas-folga/{idPermutaFolga}/rejeitar	{idPermutaFolga} (path)	Rejeitar uma permuta de folga entre funcionários

* Parâmetro opcional.

8.2 Interface do Funcionário – Módulo Horários

O WebHorariosController expõe os endpoints de consulta, geração e exportação de horários mensais, acessíveis a partir do caminho base `/web/horarios`. Para além da visualização, este controlador disponibiliza a exportação PDF e o envio de propostas ao supervisor para validação.

Endpoints do WebHorariosController (base: `/web/horarios`)

Método HTTP	Rota	Parâmetros	Objetivo
GET	/	ano* mes*	Carregar o horário mensal do utilizador e pré-formatar dados para o calendário JavaScript

(continua na página seguinte)

(continuação da tabela 14)

Método HTTP	Rota	Parâmetros	Objetivo
POST	/gerar	ano, mes	Gerar nova proposta de horário mensal (exclusivo gerente/subgerente)
GET	/exportar.pdf	ano, mes	Exportar o horário mensal em formato PDF (application/pdf, attachment)
POST	/propostas/{idProposta}/enviar	{idProposta} (path) ano*, mes*	Enviar proposta ao supervisor para validação

* Parâmetro opcional.

8.3 Arquitetura de Fluxo de Dados – UC-F01c e UC-F03

Esta secção descreve, de forma estruturada, a cadeia de invocação que ocorre em cada pedido HTTP principal associado aos casos de uso **UC-F01c** (*Preferências de Colegas de Trabalho*) e **UC-F03** (*Visualização do Calendário de Horários*). Em ambos os casos, o fluxo atravessa as seguintes camadas:

Browser → **Controller** (Spring MVC) → **BLL** (Serviços) → **Repository** (Spring Data JPA) → **PostgreSQL**

Um aspeto transversal a ambos os controladores é a **ausência de fronteira REST** entre a camada web e a BLL: os controllers invocam os mesmos *beans* Spring utilizados pela aplicação desktop, garantindo paridade de funcionalidades entre as duas plataformas.

8.3.1 UC-F01c – Preferências de Colegas de Trabalho

Carregamento da página – GET /web/complementares

Ao aceder à página de complementares, o `WebComplementaresController` executa a seguinte sequência:

1. **Verificação de sessão:** `WebAppService.obterUtilizadorIdObrigatorio(session)` resolve o `utilizadorId` a partir da sessão HTTP, ou redireciona para o login se a sessão for inválida.

2. **Verificação de permissões:** `WebAppService.obterPermissoes(utilizadorId)` determina os direitos do utilizador (`podeAprovarPreferencias`, `podeAprovarFolgas`, `podeAprovarPermutas`).

3. **Dados próprios do funcionário** (três consultas independentes):

- `PreferenciaService.listarPreferenciasPorUtilizador(id)`
→ `SELECT * FROM preferencia WHERE id_utilizador = ?`
- `DayOffService.listarPedidosPorUtilizador(id)`
→ `SELECT * FROM day_off WHERE id_utilizador = ?`
- `PermutaService.listarPedidosEnviados(id)`
→ `SELECT * FROM permuta WHERE id_utilizador_origem = ?`

4. **Filas de aprovação** (apenas se `podeAprovarPreferencias()` for verdadeiro):

- `PreferenciaService.listarPreferenciasPendentesParaAprovacao(id)`
→ preferências com estado = 'PENDENTE' da loja do utilizador
- `PreferenciaService.listarHistoricoDecisoesDaLoja(id)`
→ histórico de decisões para resolução de nomes de utilizadores

5. **Dados de suporte ao formulário:**

- `PreferenciaService.listarColegasDaLoja(id)`
→ `JOIN utilizador + loja_utilizador` para preencher a lista de colegas disponíveis
- `HorarioService.listarMeusTurnosDisponiveisParaPermuta(id)`
→ turnos futuros disponíveis para permuta

6. **Renderização:** O modelo preenchido é passado ao *template* Thymeleaf `web/complementares.html`.

Submissão de nova preferência – `POST /web/complementares/preferencias`

1. Validação do campo `tipo` contra a lista de valores permitidos: `folgas | ferias | folga_preferencia`. Uma exceção `IllegalArgumentException` é lançada para valores fora desta lista.
2. Conversão das datas opcionais com `LocalDate.parse()` (exceção em caso de formato inválido).
3. Criação da entidade `Preferencia` com estado inicial `PENDENTE`.
4. `PreferenciaService.guardarPreferencia(utilizadorId, preferencia) → PreferenciaRepository`
→ `INSERT INTO preferencia (... , estado) VALUES (... , 'PENDENTE')`.
5. Redirecionamento `POST/Redirect/GET` para `/web/complementares` com atributo *flash* de sucesso ou de erro, evitando resubmissão acidental.

Aprovação, rejeição ou remoção – `POST /preferencias/{id}/...`

O gestor invoca um dos três endpoints especializados, cada um delegando numa operação distinta da `PreferenciaService` que resulta numa instrução SQL diferente:

- **Aprovar:** UPDATE preferencia SET estado='APROVADO', id_decisor=?, data_decisao=NOW() WHERE id=?
- **Rejeitar:** UPDATE preferencia SET estado='REJEITADO', id_decisor=?, data_decisao=NOW() WHERE id=?
- **Remover:** DELETE FROM preferencia WHERE id=? AND id_utilizador=?
(apenas o próprio utilizador pode remover as suas preferências)

Em todos os casos, aplica-se o padrão *POST/Redirect/GET* com redireccionamento final para /web/complementares.

8.3.2 UC-F03 – Visualização do Calendário de Horários

Carregamento do calendário – GET /web/horarios

1. **Resolução do período:** O controlador usa os parâmetros *ano* e *mes* da *query string*, ou `LocalDate.now()` na ausência de ambos.
2. **Consulta à BLL:** `GeracaoHorariosService.obterMeusHorarios(utilizadorId, ano, mes)` executa um `SELECT` com `JOIN` entre as tabelas `horario` e `turno`, filtrado por utilizador e pelo período indicado.
3. **Projeção para o calendário JavaScript:** Cada entidade `Horario` é transformada num `Map{dia, tipo, inicio, fim}`, com as horas em formato `HH:mm`. Registos com campos nulos são descartados. Esta lista (`turnosCalendario`) é serializada pelo Thymeleaf como JSON num bloco `<script>` e consumida pelo JavaScript do lado do cliente para construir a grelha visual, evitando o risco de `LazyInitializationException` sobre *proxies* JPA dentro do *template*.
4. **Vista de gestão** (apenas se `utilizadorPodeGerarHorarios()` for verdadeiro):
 - `GeracaoHorariosService.listarPropostas(utilizadorId, ano, mes)`
→ `SELECT * FROM proposta_horario WHERE id_loja=? AND ano=? AND mes=?`
5. **Renderização:** O *template* `web/horarios.html` serve simultaneamente a vista em lista (`<table>`) e a vista em calendário (grelha JS), partilhando o mesmo modelo de dados.

Geração de proposta mensal – POST /web/horarios/gerar

1. `GeracaoHorariosService.gerarProposta(utilizadorId, ano, mes)` carrega da base de dados os funcionários ativos, regras de turnos, folgas aprovadas e preferências registadas.
2. O motor de geração (`HorarioGeneratorEngine`) executa um algoritmo de *backtracking* com restrições, atribuindo turnos de forma a respeitar limites de horas semanais, folgas obrigatórias e preferências aprovadas.
3. O resultado é persistido via `INSERT INTO proposta_horario` e `INSERT INTO horario` (um registo por turno atribuído).

4. O controlador redireciona para `GET /web/horarios?ano=&mes=` com as métricas da proposta (qualidade e pontuação) como atributo *flash*.

Exportação PDF – GET /web/horarios/exportar.pdf

1. Reutiliza `GeracaoHorariosService.obterMeusHorarios()` para obter a lista de turnos do período (nova consulta à base de dados, garantindo dados atualizados).
2. Delega na `WebPdfService`, suportada pelo Apache PDFBox, para construir o documento PDF com a grelha mensal do utilizador.
3. Devolve um `ResponseEntity<byte[]>` com os cabeçalhos `Content-Type: application/pdf` e `Content-Disposition: attachment; filename="horario-YYYY-MM.pdf"`.

8.4 Hardening de Segurança e Consistência Pós-Integração Multi-Loja

A integração da funcionalidade de multi-loja na *branch* principal exigiu um esforço adicional de verificação e correção, tanto a nível da segurança da camada web como do alinhamento entre o modelo de dados e a suite de testes de integração. Esta secção documenta as duas correções mais relevantes deste esforço.

8.4.1 Correção de Controlo de Acesso no WebGuardInterceptor

O `WebGuardInterceptor` é o componente responsável por interceptar todos os pedidos HTTP dirigidos ao prefixo `/web` e impor as regras de controlo de acesso antes que qualquer *controller* seja invocado. Para os módulos administrativos de *Gestão de Loja* e *Relatórios* – exclusivos dos perfis de Gerente/Subgerente – o método `podeAcederAoModulo(...)` deveria restringir o acesso com base no prefixo do caminho do pedido.

Detetou-se que a verificação comparava o caminho do pedido com o prefixo literal `/web/modulos/gestao` (e o equivalente para relatórios), enquanto as rotas efetivamente registadas nos *controllers* Spring MVC eram `/web/gestao-loja` e `/web/relatorios`, sem o segmento intermédio `/modulos`. Como o método `podeAcederAoModulo` devolve `true` por defeito para qualquer caminho que não corresponda explicitamente a uma das regras, esta divergência de prefixos resultava numa falha de controlo de acesso (*Broken Access Control*, OWASP Top 10 – A01:2021): qualquer colaborador autenticado, independentemente do seu cargo, conseguia contornar a verificação de permissões introduzindo diretamente o URL correto na barra de endereços, acedendo assim a módulos de gestão exclusivos da gerência.

```
1 private boolean podeAcederAoModulo(String path, Integer idUtilizador,
2     Integer idLoja) {
3     // Antes: prefixo "/web/modulos/gestao-loja" nunca correspondia
4     // a rota real "/web/gestao-loja" -> bypass por URL direto.
5     if (path.startsWith("/web/gestao-loja")) {
6         return webAppService.obterPermissoes(idUtilizador, idLoja).
7         podeGerirLoja();
8     }
9     if (path.startsWith("/web/relatorios")) {
10        return webAppService.obterPermissoes(idUtilizador, idLoja).
11        podeVerRelatorios();
12    }
13    return true;
14 }
```

Código 10: Correção do bloqueio de acesso por URL no `WebGuardInterceptor`

A correção consistiu em alinhar os prefixos verificados pelo interceptor com as rotas reais expostas pelos *controllers*, restaurando a restrição de acesso baseada em cargo (`podeGerirLoja()` / `podeVerRelatorios()`) para ambos os módulos administrativos. O impacto desta correção foi imediato: o acesso direto por URL a funcionalidades de gestão deixou de estar disponível a colaboradores sem permissões, eliminando uma via de escalonamento de privilégios na plataforma web.

8.4.2 Alinhamento de Esquema da Base de Dados e Testes de Integração

A reorganização arquitetural associada à funcionalidade multi-loja introduziu novas colunas no modelo de dados (nomeadamente `turnos.ativo` e `regras_loja.ativo`, refletindo a entidade `Turno` e a gestão de regras por loja) sem que estas alterações tivessem sido integralmente propagadas para todas as instâncias locais da base de dados PostgreSQL. Esta situação só se tornou visível ao correr a suite completa de testes de integração após a sincronização do repositório, que reportava 38 erros do tipo `InvalidDataAccessResourceUsage` com a mensagem `column "ativo" does not exist`.

A causa raiz identificada foi uma limitação conhecida do modo `ddl-auto=update` do Hibernate: embora este consiga criar tabelas e colunas anuláveis automaticamente, falha silenciosamente – sem interromper o arranque da aplicação – ao tentar adicionar uma coluna `NOT NULL` sem valor por defeito a uma tabela já populada, uma vez que o PostgreSQL rejeita esta operação para preservar a integridade dos registos existentes. O esquema da base de dados ficava, assim, permanentemente desalinhado com o modelo de entidades JPA, sem qualquer aviso explícito no momento da falha.

```
1 -- Flag de ativo/inativo nos turnos (entidade Turno.java)
2 ALTER TABLE public.turnos
3   ADD COLUMN IF NOT EXISTS ativo BOOLEAN NOT NULL DEFAULT TRUE;
```

Código 11: Migração idempotente adicionada para sincronizar o esquema com a entidade `Turno`

A resolução passou por duas ações complementares: a aplicação direta da migração em falta à base de dados local, e a atualização do script partilhado e idempotente `sql/migracao-junho2026.sql` com o passo correspondente, garantindo que qualquer outra máquina de desenvolvimento da equipa converge automaticamente para o mesmo esquema ao executar o script, sem necessidade de intervenção manual repetida.

Após a correção do esquema, restava uma única falha na suite de 151 testes: o teste `WebFluxosCriticosE2ETest.complementaresCobremFolgasPreferenciasEPermutas` submetia uma preferência com `tipo="folgas"`, um valor que deixara de constar na lista de tipos válidos (`folga_preferida`, `colegas`, `turnos`) após uma refatorização do módulo de preferências que passou a basear a folga preferida no dia da semana em vez de num intervalo de datas. O pedido HTTP continuava a devolver um código de redireção (`3xx`), mascarando o facto de a preferência nunca ter sido persistida, o que fazia falhar a asserção subsequente. A correção do valor do parâmetro no teste, sem qualquer alteração ao código de produção, restabeleceu a coerência entre o cenário de teste e o comportamento atual do sistema.

Com ambas as correções aplicadas, a suite completa de testes unitários e de integração – **151 testes** – passou a executar sem qualquer falha ou erro (`Tests run: 151, Failures: 0, Errors: 0`), confirmando a estabilidade da plataforma após a integração da funcionalidade multi-loja.

8.4.3 Gestão Multi-Loja e Consistência de Navegação: o enum `ContextoSelecao`

A introdução do suporte a múltiplas lojas (*NorteShopping* e a loja original) obrigou a repensar o ecrã de seleção de loja, partilhado entre dois pontos de entrada distintos: imediatamente após a autenticação, e a partir do painel principal, quando um gerente com acesso a mais do que uma loja decide trocar de contexto operacional. Tratar estes

dois cenários de forma indiferenciada criava dois *becos sem saída* no fluxo de navegação: um colaborador que trocasse de loja a partir do painel e depois clicasse em “voltar” era reencaminhado para o ecrã de login, perdendo a sessão activa; inversamente, tratar a chegada vinda do login como uma simples troca de loja deixava o botão de recuo sem destino válido, uma vez que ainda não existia nenhum painel para onde voltar.

A solução adotada isola esta decisão num pequeno enum dedicado, `ContextoSelecacao`, injetado no `SelecionarLojaController` no momento da inicialização e usado exclusivamente para parametrizar o comportamento do botão de recuo:

```
1 public enum ContextoSelecacao {  
2     LOGIN,           // chegada pos-autenticacao: recuar = voltar ao login  
3     DASHBOARD       // troca de loja a partir do painel: recuar = cancelar  
                       e manter sessao  
4 }
```

Código 12: Enum `ContextoSelecacao` – origem da navegação para o ecrã de seleção de loja

O `LoginController` invoca `inicializarComLigacoes(..., ContextoSelecacao.LOGIN)` após uma autenticação bem-sucedida, enquanto o `DashboardController` invoca a mesma operação com `ContextoSelecacao.DASHBOARD` sempre que o utilizador opta por “Alterar Loja” a partir da barra lateral. O próprio `SelecionarLojaController` consulta o contexto recebido tanto para decidir o texto do botão de recuo como para escolher o destino efetivo da navegação, eliminando por construção qualquer estado intermédio inconsistente entre sessão e loja ativa.

Esta gestão multi-loja é reforçada por um isolamento estrito dos dados ao nível dos serviços: cada consulta a turnos, preferências, folgas e permutas é sempre parametrizada pelo identificador da loja ativa na sessão (`idLoja`), nunca assumindo um contexto global único. Adicionalmente, a camada de configuração de loja (`GestaoLojaService`) impõe uma tranca contra duplicações de turnos: dois turnos só podem coexistir, entre os ativos e os inativos da mesma loja, se diferirem tanto no nome como no intervalo horário, prevenindo a criação accidental de configurações ambíguas ao reativar um turno previamente desativado.

8.4.4 Pipeline de Validação da Configuração de Horário da Loja

A alteração das horas de abertura e fecho de uma loja (`GestaoLojaService.guardarConfiguracao`) não pode ser uma simples atualização do registo: os turnos ativos já configurados para essa loja têm de continuar a ser compatíveis com o novo horário de funcionamento, sob pena de a aplicação ficar com turnos “órfãos” fora do intervalo de abertura, ou com períodos do dia sem qualquer colaborador escalado. O método implementa por isso um pipeline de validação em duas restrições complementares, aplicadas antes de qualquer escrita na base de dados.

Restrição de Teto (encolher o horário): ao reduzir o intervalo de funcionamento, cada turno ativo existente é confrontado com a nova janela horária; se algum turno começar antes da nova abertura ou terminar depois do novo fecho, a operação é rejeitada com uma mensagem que identifica o turno em conflito, obrigando o gestor a ajustá-lo ou desativá-lo previamente. Esta verificação impede que turnos ativos passem a representar, silenciosamente, horas fora do funcionamento real da loja.

Restrição de Piso – Cobertura Total dos Extremos: a segunda validação é a inversa da primeira e resolve o que pode ser descrito como o *paradoxo da loja vazia*: nada impede, a priori, que uma loja fique sem qualquer turno ativo a cobrir exatamente o instante de abertura ou o instante de fecho (por exemplo, se todos os turnos ativos começarem 30 minutos depois da abertura oficial). O sistema exige por isso a existência de, pelo menos, um turno ativo cujo `horaInicio` coincida exatamente com a nova abertura, e de outro cujo `horaFim` coincida exatamente com o novo fecho – sem esta cobertura nos dois extremos, a configuração é rejeitada e o gestor é informado explicitamente de qual dos dois limites (abertura, fecho, ou ambos) carece de um turno correspondente.

```
1 private boolean intervaloCorrespondeExatamente(Turno turno, LocalTime
   abertura, LocalTime fecho) {
2     return turno.isAtivo()
3         && turno.getHoraInicio().equals(abertura)
4         && turno.getHoraFim().equals(fecho);
5 }
6 // guardarConfiguracao(...) bloqueia a alteracao caso nao exista
7 // nenhum turno ativo a cobrir exatamente a abertura e/ou o fecho
   pedidos.
```

Código 13: Verificação da cobertura exata dos extremos de abertura/fecho

As duas restrições atuam em conjunto: a primeira garante que nenhum turno ativo extravasa o novo horário (teto), a segunda garante que o novo horário não fica, nos seus extremos, sem nenhum turno que o cubra (piso). Só quando ambas são satisfeitas a nova configuração é persistida, mantendo a invariante de que o horário de funcionamento declarado da loja é sempre integralmente coberto pela escala ativa.

8.4.5 Outros Aspetos de Implementação: Concorrência na UI, Tratamento de Erros e Controlo de Acesso

Para além das funcionalidades centrais, três detalhes de implementação contribuem de forma discreta, mas significativa, para a robustez da plataforma.

Execução assíncrona da geração heurística de horários: o algoritmo de *backtracking* do `HorarioGeneratorEngine` pode demorar vários segundos a convergir para uma solução válida, consoante o número de colaboradores e restrições em jogo. Para evitar bloquear a thread de interface gráfica do JavaFX durante esse cálculo – o que congelaria toda a aplicação desktop –, o `GeracaoHorariosController` delega a invocação do serviço de geração a um utilitário dedicado, o `BackgroundTaskRunner`, que encapsula a tarefa num `javafx.concurrent.Task` executado numa *thread daemon* própria (nomeada, por convenção, `geracao-horarios-worker`). O resultado é entregue de volta à *Application Thread* através dos callbacks `onSucceeded/onFailed` do próprio `Task`, garantindo que qualquer atualização da UI ocorre sempre na thread correta, e que o utilizador é impedido de disparar uma segunda geração em simultâneo enquanto a primeira ainda está em curso.

Interceção e normalização de erros de validação: as exceções de domínio lançadas pela camada de serviços (tipicamente `IllegalArgumentException` e `IllegalStateException`, usadas de forma consistente para sinalizar violações de regras de negócio como as restrições de Teto/Piso descritas acima) são capturadas junto da fronteira de apresentação – nos *controllers* JavaFX através de diálogos de erro, e na camada REST através de um `@ExceptionHandler` dedicado que as traduz num `ApiErrorResponse` estruturado com o

respetivo `HttpStatus`, em vez de deixar propagar *stack traces* ou códigos de erro genéricos até ao cliente.

Reforço de cabeçalhos de cache no `WebGuardInterceptor`: além da validação de permissões por módulo já discutida, o interceptor define explicitamente os cabeçalhos `Cache-Control: no-store, no-cache, must-revalidate`, `Pragma: no-cache` e `Expires: 0` em todas as respostas autenticadas sob `/web`. Esta medida impede que o botão “Voltar” do navegador exiba páginas sensíveis (por exemplo, dados de outro colaborador ou de uma loja diferente) a partir da cache local após um *logout*, complementando a invalidação de sessão do lado do servidor com uma garantia equivalente do lado do cliente.

9 Conclusão e Trabalho Futuro

9.1 Conclusão

O desenvolvimento deste projeto, ao longo das disciplinas de Projeto I e Projeto II, permitiu concretizar uma solução tecnológica robusta para a otimização da gestão de horários na loja Levi's Braga Parque.

A primeira fase (Projeto I) focou-se numa abordagem estruturada de engenharia de software. Partiu-se da identificação dos problemas atuais para o levantamento exaustivo de requisitos (funcionais e não funcionais priorizados pelo método MoSCoW). Procedeu-se à modelação dos processos de negócio (BPMN), à especificação de casos de uso e diagramas UML, e, finalmente, ao desenho do modelo relacional, implementado com sucesso em PostgreSQL.

A segunda fase (Projeto II) permitiu materializar os modelos desenhados numa aplicação *backend* funcional utilizando o ecossistema **Java Spring Boot**. A implementação seguiu rigorosamente uma **Arquitetura em Camadas**, separando de forma clara o mapeamento de dados (JPA), as transações com a base de dados (DAL) e as regras de negócio complexas (BLL). O desenvolvimento de uma solução desktop funcional para apoio à gestão de horários por turnos, cobrindo não apenas a persistência e a lógica de negócio, mas também a operação diária dos diferentes perfis da loja.

Conclui-se que a estrutura de dados, o código desenvolvido e a documentação técnica oferecem uma base sólida e funcional, respondendo eficazmente ao objetivo de reduzir a carga administrativa e aumentar a transparência na gestão da equipa de colaboradores.

9.2 Trabalho Futuro

Apesar de a aplicação desktop se encontrar funcional e demonstrável, continuam a existir oportunidades de evolução. Como trabalho futuro, identificam-se as seguintes linhas de trabalho futuro:

1. **Plataforma Web e REST API (Entrega de Junho):** Transição e expansão do sistema para o ambiente Web. Os métodos atualmente presentes na BLL serão expostos através de controladores (*Controllers*) numa API RESTful, permitindo o acesso remoto dos funcionários através de qualquer navegador em tempo real.
2. **Algoritmo de Geração Automática:** Expandir a BLL com um algoritmo inteligente que utilize os dados parametrizados na base de dados para gerar as escalas automaticamente, respeitando todas as restrições legais e preferências dos funcionários de forma autónoma.
3. **Sistema de Notificações Automáticas:** Integração de um serviço de envio de emails ou SMS (ex: SendGrid ou Twilio) para notificar os funcionários sempre que um horário for publicado ou um pedido de permuta for atualizado.

Estas melhorias permitirão elevar o sistema de um protótipo *backend* (focado em DAL e BLL) para uma plataforma integral, moderna e multiplataforma de gestão de recursos humanos.